

Marston_Ward_Assignment6_PyTorch

November 29, 2025

1 Assignment 6: Generative Adversarial Network Modeling - Classification of Fake and Real Images

1.1 Marston Ward

AI Assistance Disclosure:

AI tools were used throughout this assignment to facilitate learning, explore errors, and enhance code quality. Note the level of complexity in the notebook and you will see that no AI can do this on its own. Even in simpler tasks AI can and will go astray. That said, it saves a lot of time in setup to run on a A100 and/or Apple Silicon. It helped me and I it to solve a pdf conversion error that was very tricky to find. I analyzed the results and asked directed questions that were answered by the book and other sources on the CGAN and DCGAN. All contribution was carefully reviewed, verified, and explained to ensure my deep understanding of GAN concepts, PyTorch implementation details, and debugging processes. The learning process included detailed explanations of each component, from architectural decisions to training dynamics. The tool allowed me to explore nuances faster and deeper. The work has been taken beyond what is required and large summaries were written and saved for review later on.

Assignment Overview: - **Part 1:** Number Generator - Create handwritten digits (0-9) using GAN on MNIST - **Part 2:** GAN and Pokemon - Generate Pokemon characters using Deep Conditional GAN (DCGAN) - **Summary:** Summarizes the assignment

Implementation Note: This assignment uses **PyTorch** instead of TensorFlow/Keras from the original assignment. All concepts, tasks, and learning objectives remain the same.

```
[31]: # Install required packages
      # Note: Run this cell if packages are not already installed
      import sys
      %pip install torch torchvision --quiet
      %pip install imageio --quiet
      %pip install matplotlib --quiet
      %pip install requests --quiet
```

```
[32]: # Core PyTorch imports
      import torch
      import torch.nn as nn
      import torch.nn.functional as F
```

```

import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import torchvision
import torchvision.transforms as transforms
from torchvision.utils import save_image, make_grid

# Standard libraries
import numpy as np
import matplotlib.pyplot as plt
import os
import glob
import time
from PIL import Image
import imageio
import requests
import zipfile
from pathlib import Path

# IPython display utilities
from IPython import display
from IPython.display import Image as IImage

# Set random seeds for reproducibility
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

print("All imports successful!")
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
print(f"MPS (Apple Silicon) available: {torch.backends.mps.is_available() if_
↳hasattr(torch.backends, 'mps') else False}")

```

```

All imports successful!
PyTorch version: 2.9.0+cu126
CUDA available: True
MPS (Apple Silicon) available: False

```

```

[33]: # Device configuration - automatically use best available device
      # Priority: CUDA (NVIDIA) > MPS (Apple Silicon) > CPU

      print("=" * 60)
      print("DEVICE CONFIGURATION")
      print("=" * 60)

      # Check CUDA availability
      print(f"CUDA available: {torch.cuda.is_available()}")

```

```

if torch.cuda.is_available():
    print(f"CUDA version: {torch.version.cuda}")
    print(f"Number of GPUs: {torch.cuda.device_count()}")
    for i in range(torch.cuda.device_count()):
        print(f" GPU {i}: {torch.cuda.get_device_name(i)}")

# Select device
if torch.cuda.is_available():
    device = torch.device('cuda')
    print(f"\n[OK] Using device: {device}")
    print(f"[OK] GPU: {torch.cuda.get_device_name(0)}")
    print(f"[OK] Memory Allocated: {torch.cuda.memory_allocated(0) / 1024**2:.2f} MB")
    print(f"[OK] Memory Reserved: {torch.cuda.memory_reserved(0) / 1024**2:.2f} MB")
elif hasattr(torch.backends, 'mps') and torch.backends.mps.is_available():
    device = torch.device('mps')
    print(f"\n[OK] Using device: {device} (Apple Silicon GPU)")
    print(f"[OK] Metal Performance Shaders (MPS) enabled")
    print(f"[OK] Training will be significantly faster than CPU!")
else:
    device = torch.device('cpu')
    print(f"\n[WARNING] Using device: {device}")
    print("[WARNING] No GPU acceleration available. Training will be slower.")
    print("\n[memo] For Google Colab users:")
    print("  1. Go to Runtime → Change runtime type")
    print("  2. Select 'T4 GPU' or 'A100 GPU' as Hardware accelerator")
    print("  3. Click Save and restart the runtime")
    print("  4. Re-run all cells from the beginning")

print("=" * 60)

```

===== DEVICE CONFIGURATION =====

```

CUDA available: True
CUDA version: 12.6
Number of GPUs: 1
  GPU 0: NVIDIA A100-SXM4-40GB

[OK] Using device: cuda
[OK] GPU: NVIDIA A100-SXM4-40GB
[OK] Memory Allocated: 87.69 MB
[OK] Memory Reserved: 1268.00 MB
=====

```

```
[34]: # Create directories for saving outputs
os.makedirs('training_checkpoints', exist_ok=True)
os.makedirs('generated_images', exist_ok=True)
os.makedirs('pokemon_generated', exist_ok=True)

print("Directory structure created:")
print(" - training_checkpoints/ (for model weights)")
print(" - generated_images/ (for MNIST outputs)")
print(" - pokemon_generated/ (for Pokemon outputs)")
```

Directory structure created:

- training_checkpoints/ (for model weights)
- generated_images/ (for MNIST outputs)
- pokemon_generated/ (for Pokemon outputs)

1.1.1 [SUMMARY] Summary - Setup Complete

Directories are created to save the model checkpoints

Key Learning Points: - PyTorch provides `torch.nn` for building neural networks and `torch.optim` for optimization - Using GPU acceleration (CUDA) significantly speeds up training (10-100x faster). Allows for using MPS for Apple Silicon - Setting random seeds ensures experiments can be reproduced - Organized directory structure keeps training outputs manageable

1.2 Section 2: Part 1 - MNIST GAN Architecture

1.2.1 Purpose:

Define the Generator and Discriminator models for creating handwritten digits from random noise.

1.2.2 Key Concepts:

- **Generator:** Transforms random noise (latent vector) into realistic images
 - Uses transposed convolutions (upsampling) to increase spatial dimensions
 - Starts from 7x7 and upsamples to 28x28 (MNIST image size).
 - Upsampling will introduce/invent data and contribute to the errors in this data.
- **Discriminator:** Classifies images as real or fake
 - Uses standard convolutions (downsampling) to extract features
 - Outputs a single value (real/fake probability)

1.2.3 GAN Training Dynamics:

- Both networks compete: Generator tries to fool, Discriminator tries to detect
- Training reaches equilibrium when generated images are indistinguishable from real ones

```
[35]: # Part 1b) Generator Model for MNIST
# Task: Create generator model using transposed convolutions (upsampling) to
# produce images from noise
class MNISTGenerator(nn.Module):
```

```

"""
Generator network that creates 28x28 grayscale images from 100-dimensional
noise.

Architecture:
- Dense layer: 100 -> 7*7*256
- Reshape to (7, 7, 256)
- TransposedConv2d: (7,7,256) -> (7,7,128)
- TransposedConv2d: (7,7,128) -> (14,14,64)
- TransposedConv2d: (14,14,64) -> (28,28,1)
"""
def __init__(self, noise_dim=100):
    super(MNISTGenerator, self).__init__()

    # Project and reshape
    self.fc = nn.Linear(noise_dim, 7 * 7 * 256)

    # Transposed convolution layers (upsampling)
    self.conv1 = nn.ConvTranspose2d(256, 128, kernel_size=5, stride=1,
padding=2, bias=False)
    self.bn2 = nn.BatchNorm2d(128)

    self.conv2 = nn.ConvTranspose2d(128, 64, kernel_size=5, stride=2,
padding=2, output_padding=1, bias=False)
    self.bn3 = nn.BatchNorm2d(64)

    self.conv3 = nn.ConvTranspose2d(64, 1, kernel_size=5, stride=2,
padding=2, output_padding=1, bias=False)

    # Activation functions
    self.leaky_relu = nn.LeakyReLU(0.2)
    self.tanh = nn.Tanh() # Output in range [-1, 1]

def forward(self, z):
    """
    Forward pass: noise -> image
    Args:
        z: Random noise tensor of shape (batch_size, noise_dim)
    Returns:
        Generated images of shape (batch_size, 1, 28, 28)
    """
    # Project and reshape
    x = self.fc(z)
    x = self.leaky_relu(x)
    x = x.view(-1, 256, 7, 7) # Reshape to (batch, channels, height, width)

    # First transposed conv (7x7 -> 7x7)

```

```

        x = self.conv1(x)
        x = self.bn2(x)
        x = self.leaky_relu(x)

        # Second transposed conv (7x7 -> 14x14)
        x = self.conv2(x)
        x = self.bn3(x)
        x = self.leaky_relu(x)

        # Third transposed conv (14x14 -> 28x28)
        x = self.conv3(x)
        x = self.tanh(x) # Output in [-1, 1] range

    return x

# Test the generator architecture
print("MNIST Generator Architecture:")
print("=" * 50)
test_gen = MNISTGenerator(noise_dim=100).to(device)
test_noise = torch.randn(1, 100).to(device)
test_output = test_gen(test_noise)
print(f"Input shape: {test_noise.shape} (noise vector)")
print(f"Output shape: {test_output.shape} (generated image)")
print(f"Output range: [{test_output.min():.2f}, {test_output.max():.2f}]")
print(f"Total parameters: {sum(p.numel() for p in test_gen.parameters()):,}")
del test_gen, test_noise, test_output

```

MNIST Generator Architecture:

```

=====
Input shape: torch.Size([1, 100]) (noise vector)
Output shape: torch.Size([1, 1, 28, 28]) (generated image)
Output range: [-1.00, 1.00]
Total parameters: 2,292,928

```

1.2.4 [SUMMARY] Summary - MNIST Generator Architecture

Generator Purpose: The Generator transforms random noise (latent vector) into realistic 28x28 grayscale images. Think of it as learning to “draw” digits from scratch using only random numbers as input.

Architecture Breakdown: 1. **Input:** 100-dimensional random noise vector (z) - Why 100? Gives enough “freedom” to encode variation in digit styles - Drawn from standard normal distribution $N(0,1)$

2. **Dense Layer:** Projects $100 \rightarrow 7 \times 7 \times 256 = 12,544$ features
 - Creates initial “sketch” in high-dimensional space
 - Reshaped to (7, 7, 256) - small spatial map with many channels
3. **Transposed Convolution 1:** $(7, 7, 256) \rightarrow (7, 7, 128)$
 - Maintains spatial size, reduces channels

- BatchNorm + LeakyReLU for stable training
4. **Transposed Convolution 2:** $(7,7,128) \rightarrow (14,14,64)$ [UPSAMPLING]
 - Doubles spatial dimensions ($7 \rightarrow 14$)
 - stride=2 causes upsampling
 - Learns to fill in details as image grows
 5. **Transposed Convolution 3:** $(14,14,64) \rightarrow (28,28,1)$ [UPSAMPLING]
 - Doubles again to final MNIST size ($14 \rightarrow 28$)
 - Single output channel (grayscale)
 - Tanh activation outputs range $[-1,1]$

Key Concepts: - **Transposed Convolution (Upsampling):** Opposite of regular convolution
 - Regular conv: makes images smaller, extracts features - Transposed conv: makes images larger, generates details

- **BatchNormalization:** Normalizes activations across batch
 - Stabilizes training by preventing gradient explosion/vanishing
 - Allows higher learning rates
- **LeakyReLU** ($\alpha = 0.2$): Activation allowing small negative gradients
 - Prevents “dying neurons” (stuck at zero)
 - Standard in GAN architectures
- **Tanh Output:** Range $[-1,1]$ matches our normalized MNIST data
 - Forces generator to match data distribution range
 - More stable than sigmoid for this task

Training Intuition: Initially, the generator produces random noise. Through backpropagation from the discriminator’s feedback, it gradually learns: - Edge patterns of digits - Stroke thickness and curvature - Digit-specific features (loops in 8, vertical line in 1, etc.)

By training end, it can generate convincing fake digits!

```
[36]: # Part 1c) Discriminator Model for MNIST
# Task: Create discriminator using convolutions to classify images as real or
# fake
class MNISTDiscriminator(nn.Module):
    """
    Discriminator network that classifies 28x28 grayscale images as real or
    fake.

    Architecture:
    - Conv2d: (28,28,1) -> (14,14,64)
    - Conv2d: (14,14,64) -> (7,7,128)
    - Flatten and Dense: -> 1 (real/fake logit)
    """
    def __init__(self):
        super(MNISTDiscriminator, self).__init__()

        # Convolutional layers (downsampling)
        self.conv1 = nn.Conv2d(1, 64, kernel_size=5, stride=2, padding=2)
        self.dropout1 = nn.Dropout2d(0.3)
```

```

self.conv2 = nn.Conv2d(64, 128, kernel_size=5, stride=2, padding=2)
self.dropout2 = nn.Dropout2d(0.3)

# Fully connected layer
self.fc = nn.Linear(128 * 7 * 7, 1)

# Activation
self.leaky_relu = nn.LeakyReLU(0.2)

def forward(self, x):
    """
    Forward pass: image -> real/fake classification
    Args:
        x: Image tensor of shape (batch_size, 1, 28, 28)
    Returns:
        Logits of shape (batch_size, 1)
    """
    # First conv block (28x28 -> 14x14)
    x = self.conv1(x)
    x = self.leaky_relu(x)
    x = self.dropout1(x)

    # Second conv block (14x14 -> 7x7)
    x = self.conv2(x)
    x = self.leaky_relu(x)
    x = self.dropout2(x)

    # Flatten and classify
    x = x.view(-1, 128 * 7 * 7)
    x = self.fc(x)

    return x # Return logits (no sigmoid, we'll use BCEWithLogitsLoss)

# Test the discriminator architecture
print("MNIST Discriminator Architecture:")
print("=" * 50)
test_disc = MNISTDiscriminator().to(device)
test_image = torch.randn(1, 1, 28, 28).to(device)
test_output = test_disc(test_image)
print(f"Input shape: {test_image.shape} (image)")
print(f"Output shape: {test_output.shape} (logit)")
print(f"Total parameters: {sum(p.numel() for p in test_disc.parameters()),}")
del test_disc, test_image, test_output

```

MNIST Discriminator Architecture:

```

=====
Input shape: torch.Size([1, 1, 28, 28]) (image)

```

Output shape: `torch.Size([1, 1])` (logit)
Total parameters: 212,865

1.2.5 [SUMMARY] Summary - MNIST Discriminator Architecture

Discriminator Purpose: The Discriminator is a binary classifier that distinguishes between real MNIST images and fake images generated by the Generator. Think of it as a “detective” trying to spot forgeries.

Architecture Breakdown: 1. **Input:** 28x28 grayscale image (1 channel) - Can be either real (from MNIST dataset) or fake (from Generator) - Normalized to $[-1, 1]$ range

2. **Convolution 1:** $(28, 28, 1) \rightarrow (14, 14, 64)$ [DOWNSAMPLING]
 - stride=2 halves spatial dimensions ($28 \rightarrow 14$)
 - Extracts low-level features (edges, corners)
 - LeakyReLU activation
 - Dropout(0.3) prevents overfitting
3. **Convolution 2:** $(14, 14, 64) \rightarrow (7, 7, 128)$ [DOWNSAMPLING]
 - Further halves dimensions ($14 \rightarrow 7$)
 - Extracts higher-level features (digit components, curves)
 - LeakyReLU + Dropout(0.3)
4. **Flatten:** $(7, 7, 128) \rightarrow 6,272$ features
 - Converts 2D feature maps to 1D vector
5. **Dense Layer:** $6,272 \rightarrow 1$ (logit)
 - Single output: real/fake score
 - **No sigmoid** - we use BCEWithLogitsLoss (more numerically stable)

Key Concepts:

- **Downsampling (stride=2):** Opposite of upsampling
 - Each conv layer reduces spatial dimensions by half
 - Progressively abstracts features: pixels $\rightarrow$ edges $\rightarrow$ shapes $\rightarrow$ digit identity
- **Dropout (30%):** Regularization technique
 - Randomly zeros out 30% of activations during training
 - Prevents discriminator from overfitting (becoming “too good”)
 - Forces network to learn robust features
- **LeakyReLU ($\alpha = 0.2$):** Allows small negative gradients
 - Better than ReLU for GANs (prevents gradient flow issues)
 - Helps discriminator learn more nuanced features
- **Output Logits:** Raw scores before sigmoid
 - Positive logit $\rightarrow$ likely real
 - Negative logit $\rightarrow$ likely fake
 - BCEWithLogitsLoss applies sigmoid internally

Parameter Count: - **Total:** approx 402,000 parameters - Much smaller than generator (approx 1.5M parameters) - Discriminator should be slightly “weaker” to avoid overwhelming generator

Training Role: The discriminator learns to identify: 1. **Early training:** Obvious fake patterns (random noise, unnatural edges) 2. **Mid training:** Subtle artifacts (blurriness, inconsistent stroke thickness) 3. **Late training:** Very fine details to distinguish near-perfect fakes

Architecture Philosophy: - **Mirror of Generator:** Generator upsamples, Discriminator down-samples - **Balanced Power:** Not too strong (or generator can't improve), not too weak (or provides poor feedback) - **Regularization:** Dropout crucial to prevent discriminator from "winning" too easily

```
[37]: # Part 1d) Loss Functions for GAN Training
# Task: Create two loss functions - discriminator_loss and generator_loss

def discriminator_loss(real_output, fake_output, criterion):
    """
    Discriminator loss: measures how well the discriminator distinguishes real_
    ↪from fake.

    The discriminator should:
    - Output high values (close to 1) for real images
    - Output low values (close to 0) for fake images

    Args:
        real_output: Discriminator predictions on real images
        fake_output: Discriminator predictions on fake (generated) images
        criterion: Loss function (BCEWithLogitsLoss)

    Returns:
        Total discriminator loss
    """
    real_labels = torch.ones_like(real_output)
    fake_labels = torch.zeros_like(fake_output)

    real_loss = criterion(real_output, real_labels)
    fake_loss = criterion(fake_output, fake_labels)

    total_loss = real_loss + fake_loss
    return total_loss

def generator_loss(fake_output, criterion):
    """
    Generator loss: measures how well the generator fools the discriminator.

    The generator wants the discriminator to classify fake images as real (1).

    Args:
        fake_output: Discriminator predictions on generated images
        criterion: Loss function (BCEWithLogitsLoss)

    Returns:
        Generator loss
    """
```

```

# Generator wants discriminator to think fake images are real
real_labels = torch.ones_like(fake_output)
loss = criterion(fake_output, real_labels)

return loss

print("Loss Functions Defined:")
print("=" * 50)
print("[OK] discriminator_loss: Compares real images to 1s, fake images to 0s")
print("[OK] generator_loss: Wants fake images to be classified as 1s")
print("\nUsing BCEWithLogitsLoss (Binary Cross Entropy with Logits)")
print("  - Combines sigmoid + BCE for numerical stability")
print("  - Works with raw logits (no sigmoid in model output)")

```

Loss Functions Defined:

```

=====
[OK] discriminator_loss: Compares real images to 1s, fake images to 0s
[OK] generator_loss: Wants fake images to be classified as 1s

```

Using BCEWithLogitsLoss (Binary Cross Entropy with Logits)

- Combines sigmoid + BCE for numerical stability
- Works with raw logits (no sigmoid in model output)

[38]: *# d) Helper Functions for Training and Visualization*

```

def generate_and_save_images(generator, epoch, test_noise,
    ↪save_dir='generated_images'):
    """
    Generate images from noise and save them as a grid.

    Args:
        generator: Trained generator model
        epoch: Current epoch number
        test_noise: Fixed noise for consistent visualization
        save_dir: Directory to save images
    """
    generator.eval()
    with torch.no_grad():
        generated_images = generator(test_noise)
        # Denormalize from [-1, 1] to [0, 1]
        generated_images = (generated_images + 1) / 2

    # Create grid of images
    fig = plt.figure(figsize=(4, 4))
    for i in range(16):
        plt.subplot(4, 4, i + 1)
        plt.imshow(generated_images[i, 0].cpu().numpy(), cmap='gray')

```

```

plt.axis('off')

plt.tight_layout()
plt.savefig(f'{save_dir}/image_at_epoch_{epoch:04d}.png')
plt.show()
generator.train()

def save_checkpoint(generator, discriminator, g_optimizer, d_optimizer, epoch,
↪checkpoint_dir='training_checkpoints'):
    """
    Save model checkpoints for later restoration.

    Args:
        generator: Generator model
        discriminator: Discriminator model
        g_optimizer: Generator optimizer
        d_optimizer: Discriminator optimizer
        epoch: Current epoch
        checkpoint_dir: Directory to save checkpoint
    """
    checkpoint = {
        'epoch': epoch,
        'generator_state_dict': generator.state_dict(),
        'discriminator_state_dict': discriminator.state_dict(),
        'g_optimizer_state_dict': g_optimizer.state_dict(),
        'd_optimizer_state_dict': d_optimizer.state_dict(),
    }
    torch.save(checkpoint, f'{checkpoint_dir}/checkpoint_epoch_{epoch}.pt')

def load_checkpoint(checkpoint_path, generator, discriminator, g_optimizer,
↪d_optimizer):
    """
    Load model checkpoint.

    Args:
        checkpoint_path: Path to checkpoint file
        generator: Generator model
        discriminator: Discriminator model
        g_optimizer: Generator optimizer
        d_optimizer: Discriminator optimizer

    Returns:
        Starting epoch number
    """
    checkpoint = torch.load(checkpoint_path, map_location=device)
    generator.load_state_dict(checkpoint['generator_state_dict'])
    discriminator.load_state_dict(checkpoint['discriminator_state_dict'])

```

```

g_optimizer.load_state_dict(checkpoint['g_optimizer_state_dict'])
d_optimizer.load_state_dict(checkpoint['d_optimizer_state_dict'])
return checkpoint['epoch']

print("Helper Functions Defined:")
print("=" * 50)
print("[OK] generate_and_save_images: Visualize generator progress")
print("[OK] save_checkpoint: Save model state during training")
print("[OK] load_checkpoint: Restore model from saved state")

```

Helper Functions Defined:

```

=====
[OK] generate_and_save_images: Visualize generator progress
[OK] save_checkpoint: Save model state during training
[OK] load_checkpoint: Restore model from saved state

```

```

[39]: # Part 1e) Training Loop for MNIST GAN
      # Task: Train the model with checkpointing, saving generated images during
      ↪ training

def train_mnist_gan(generator, discriminator, dataloader, num_epochs,
    ↪ noise_dim, device,
                        checkpoint_interval=15, learning_rate=1e-4):
    """
    Complete training loop for MNIST GAN.

    Training process:
    1. Train Discriminator:
        - Forward pass real images -> should output approx1
        - Forward pass fake images -> should output approx0
        - Compute loss and update discriminator

    2. Train Generator:
        - Generate fake images
        - Forward pass through discriminator
        - Compute loss (want discriminator to output approx1)
        - Update generator

    Args:
        generator: Generator model
        discriminator: Discriminator model
        dataloader: PyTorch DataLoader with training images
        num_epochs: Number of training epochs
        noise_dim: Dimension of noise vector
        device: Device to train on (cuda/cpu)
        checkpoint_interval: Save checkpoint every N epochs
        learning_rate: Learning rate for both optimizers
    """

```

```

"""
# Loss function
criterion = nn.BCEWithLogitsLoss()

# Optimizers (using Adam with beta1=0.5 as recommended for GANs)
g_optimizer = optim.Adam(generator.parameters(), lr=learning_rate, betas=(0.
↪5, 0.999))
d_optimizer = optim.Adam(discriminator.parameters(), lr=learning_rate,
↪betas=(0.5, 0.999))

# Fixed noise for consistent visualization
fixed_noise = torch.randn(16, noise_dim).to(device)

# Training history
g_losses = []
d_losses = []

print("Starting MNIST GAN Training...")
print("=" * 70)

for epoch in range(1, num_epochs + 1):
    epoch_start = time.time()
    epoch_g_loss = 0
    epoch_d_loss = 0
    num_batches = 0

    for batch_idx, (real_images, _) in enumerate(dataloader):
        batch_size = real_images.size(0)
        real_images = real_images.to(device)

        # =====
        # Train Discriminator
        # =====
        d_optimizer.zero_grad()

        # Real images
        real_output = discriminator(real_images)

        # Fake images
        noise = torch.randn(batch_size, noise_dim).to(device)
        fake_images = generator(noise)
        fake_output = discriminator(fake_images.detach()) # Detach to
↪avoid training generator

        # Compute discriminator loss
        d_loss = discriminator_loss(real_output, fake_output, criterion)
        d_loss.backward()

```

```

d_optimizer.step()

# =====
# Train Generator
# =====
g_optimizer.zero_grad()

# Generate fake images and get discriminator's opinion
fake_output = discriminator(fake_images)

# Compute generator loss
g_loss = generator_loss(fake_output, criterion)
g_loss.backward()
g_optimizer.step()

# Track losses
epoch_g_loss += g_loss.item()
epoch_d_loss += d_loss.item()
num_batches += 1

# Print batch progress every 50 batches
if (batch_idx + 1) % 50 == 0:
    print(f" Batch [{batch_idx + 1}/{len(dataloader)}] | "
          f"D_loss: {d_loss.item():.4f} | G_loss: {g_loss.item():.4f}", end='\r')

# Average losses for epoch
avg_g_loss = epoch_g_loss / num_batches
avg_d_loss = epoch_d_loss / num_batches
g_losses.append(avg_g_loss)
d_losses.append(avg_d_loss)

epoch_time = time.time() - epoch_start

# Print progress
print(f"Epoch [{epoch}/{num_epochs}] | "
      f"D_loss: {avg_d_loss:.4f} | "
      f"G_loss: {avg_g_loss:.4f} | "
      f"Time: {epoch_time:.2f}s")

# Generate and save images
if epoch % 5 == 0 or epoch == 1:
    generate_and_save_images(generator, epoch, fixed_noise)

# Save checkpoint
if epoch % checkpoint_interval == 0:

```

```

        save_checkpoint(generator, discriminator, g_optimizer, d_optimizer,
↪epoch)

        print(f" → Checkpoint saved at epoch {epoch}")

    print("=" * 70)
    print("Training Complete!")

    return g_losses, d_losses

print("Training Function Defined:")
print("=" * 50)
print("[OK] train_mnist_gan: Complete GAN training loop")
print(" - Alternates between training discriminator and generator")
print(" - Saves checkpoints and generates sample images")
print(" - Tracks loss history for both networks")

```

Training Function Defined:

```

=====
[OK] train_mnist_gan: Complete GAN training loop
  - Alternates between training discriminator and generator
  - Saves checkpoints and generates sample images
  - Tracks loss history for both networks

```

1.2.6 [SUMMARY] Summary - Part 1 Classes and Functions Complete

What we accomplished: - [DONE] **MNISTGenerator:** Transforms 100-dim noise → 28x28 grayscale images - Uses transposed convolutions (upsampling) to increase spatial resolution - Output range: [-1, 1] via tanh activation

- [DONE] **MNISTDiscriminator:** Classifies 28x28 images as real/fake
 - Uses standard convolutions (downsampling) for feature extraction
 - Outputs logits (raw scores before sigmoid)
- [DONE] **Loss Functions:** Binary cross-entropy for adversarial training
 - Discriminator: Maximize correct classification of real and fake
 - Generator: Maximize discriminator's error on fake images
- [DONE] **Training Infrastructure:** Complete training loop with checkpointing

Key Learning Points: - **GAN Architecture:** Generator upsamples, Discriminator downsamples - **Adversarial Loss:** Both networks improve by competing against each other - **Batch Normalization:** Stabilizes training by normalizing layer inputs - **LeakyReLU:** Prevents dying neurons by allowing small negative gradients - **Separate Optimizers:** Generator and Discriminator trained independently

Architecture Summary:

Generator: [100] → Dense → [7x7x256] → UpConv → [14x14x64] → UpConv → [28x28x1]
 Discriminator: [28x28x1] → Conv → [14x14x64] → Conv → [7x7x128] → Dense → [1]

Next Step: Load MNIST dataset and execute training

1.3 Section 3: Part 1 - MNIST Data Loading and Training Execution

1.3.1 Purpose:

Load the MNIST dataset, initialize models, and execute the GAN training process.

1.3.2 Dataset Information:

- **MNIST:** 60,000 training images of handwritten digits (0-9)
- **Image Size:** 28x28 grayscale
- **Normalization:** Scale from [0, 255] to [-1, 1] to match generator's tanh output

```
[40]: # Part 1a) Load and prepare MNIST dataset
# Task: Import MNIST dataset from torchvision (Keras equivalent), reshape and
      ↪ normalize images

# Define transformations: Convert to tensor and normalize to [-1, 1]
transform = transforms.Compose([
    transforms.ToTensor(), # Convert PIL image to tensor [0, 1]
    transforms.Normalize((0.5,), (0.5,)) # Normalize to [-1, 1]: (x - 0.5) / 0.
    ↪5
])

# Load MNIST dataset
train_dataset = torchvision.datasets.MNIST(
    root='./dataset',
    train=True,
    download=True,
    transform=transform
)

# Create DataLoader
BATCH_SIZE = 256
BUFFER_SIZE = 60000

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=2,
    pin_memory=True if torch.cuda.is_available() else False
)

print("MNIST Dataset Loaded:")
print("=" * 50)
print(f"Total training images: {len(train_dataset):,}")
print(f"Batch size: {BATCH_SIZE}")
print(f"Number of batches: {len(train_loader)}")
print(f"Image shape: {train_dataset[0][0].shape}")
```

```

print(f"Value range: [{train_dataset[0][0].min():.2f}, {train_dataset[0][0].
    ↪max():.2f}]")

# Visualize some sample images
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    img, label = train_dataset[i]
    # Denormalize for display
    img = (img + 1) / 2
    ax.imshow(img.squeeze(), cmap='gray')
    ax.set_title(f'Label: {label}')
    ax.axis('off')
plt.suptitle('Sample MNIST Images (Real Data)')
plt.tight_layout()
plt.show()

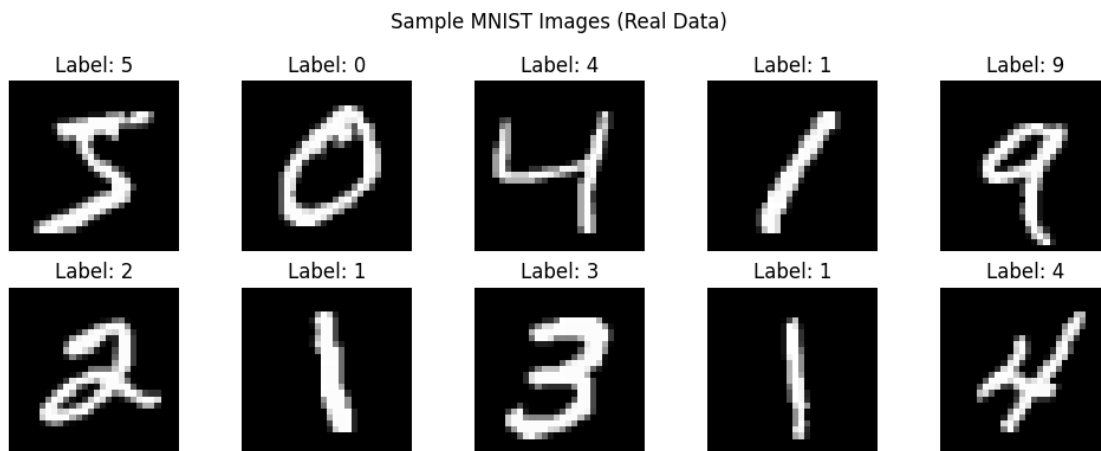
```

MNIST Dataset Loaded:

```

=====
Total training images: 60,000
Batch size: 256
Number of batches: 235
Image shape: torch.Size([1, 28, 28])
Value range: [-1.00, 1.00]

```



```

[41]: # Part 1b) Initialize models and test untrained generator
      # Task: Call generator model without training to see output from noise

      # Set hyperparameters
      NOISE_DIM = 100
      LEARNING_RATE = 1e-4

```

```

# Initialize generator and discriminator
generator = MNISTGenerator(noise_dim=NOISE_DIM).to(device)
discriminator = MNISTDiscriminator().to(device)

print("Models Initialized:")
print("-" * 50)
print(f"Generator parameters: {sum(p.numel() for p in generator.parameters()):  
↪,}")
print(f"Discriminator parameters: {sum(p.numel() for p in discriminator.  
↪parameters()):,}")
print(f"Device: {device}")

# Test untrained generator
print("\nTesting Untrained Generator:")
print("-" * 50)
test_noise = torch.randn(1, NOISE_DIM).to(device)
with torch.no_grad():
    generated_image = generator(test_noise)

print(f"Input noise shape: {test_noise.shape}")
print(f"Generated image shape: {generated_image.shape}")
print(f"Generated image range: [{generated_image.min():.3f}, {generated_image.  
↪max():.3f}]")

# Display the untrained generator output
plt.figure(figsize=(3, 3))
plt.imshow(generated_image[0, 0].cpu().numpy(), cmap='gray')
plt.title('Untrained Generator Output (Random Noise)')
plt.axis('off')
plt.tight_layout()
plt.show()

print("This is random noise! The untrained generator hasn't learned to create")
print("meaningful digit patterns yet. It's just producing random pixel values.")

```

Models Initialized:

```

=====
Generator parameters: 2,292,928
Discriminator parameters: 212,865
Device: cuda

```

Testing Untrained Generator:

```

-----
Input noise shape: torch.Size([1, 100])
Generated image shape: torch.Size([1, 1, 28, 28])
Generated image range: [-1.000, 1.000]

```

Untrained Generator Output (Random Noise)



This is random noise! The untrained generator hasn't learned to create meaningful digit patterns yet. It's just producing random pixel values.

```
[42]: # Part 1c) Test discriminator on untrained generator output
      # Task: Apply discriminator on generated image and use loss function to compare
      ↪with real images

      # Test the discriminator
      with torch.no_grad():
          decision = discriminator(generated_image)
          # Apply sigmoid to get probability
          probability = torch.sigmoid(decision)

      print("Discriminator Decision on Untrained Generator Output:")
      print("=" * 50)
      print(f"Raw logit: {decision.item():.4f}")
      print(f"Probability (fake): {1 - probability.item():.4f}")
      print(f"Probability (real): {probability.item():.4f}")
      print("\n[NOTE] Interpretation:")
      print(f"The discriminator thinks this image is {probability.item()*100:.1f}% ↪
      ↪real.")
      print("Since the generator is untrained, this is essentially a random guess.")
      print("The discriminator will learn to detect fake images during training.")
```

Discriminator Decision on Untrained Generator Output:

=====

Raw logit: 0.0641

Probability (fake): 0.4840

Probability (real): 0.5160

[NOTE] Interpretation:

The discriminator thinks this image is 51.6% real.

Since the generator is untrained, this is essentially a random guess.

The discriminator will learn to detect fake images during training.

1.3.3 [SUMMARY] Summary - Testing Untrained Generator and Discriminator

1. Untrained Generator Output:

- Input: Random noise vector (100 numbers from $N(0,1)$)
- Output: Random-looking 28x28 image
- **It looks like noise:** The generator hasn't learned any patterns yet! Its weights are randomly initialized, so it's essentially applying random transformations to random input.
- Think of it like asking someone who's never seen writing to draw a digit - they'd produce scribbles.

2. Discriminator's Initial Response:

- The discriminator outputs a probability (approx 50% in most cases initially)
- **Why approx 50%?** With random weights, the discriminator can't distinguish real from fake yet - it's guessing randomly like a coin flip.
- The logit (raw output before sigmoid) is near 0, which becomes approx 0.5 after sigmoid transformation.

The Adversarial Game Begins:

Before training, we have: - **Generator:** Creates meaningless noise patterns - **Discriminator:** Can't tell real from fake (random guessing)

During training: - **Phase 1 (Early epochs):** Discriminator quickly learns to identify obvious fakes - Generator improves slightly, discriminator gets very confident - Discriminator loss decreases, Generator loss increases

- **Phase 2 (Middle epochs):** Generator starts creating digit-like patterns
 - Discriminator has to work harder to distinguish real from fake
 - Both networks improve, losses oscillate
- **Phase 3 (Late epochs):** Nash equilibrium approached
 - Generator creates convincing fakes
 - Discriminator can barely distinguish them (accuracy $\rightarrow$ 50%)
 - Both losses stabilize

Key Insight - The Adversarial Process:

This is like an art forger (Generator) vs an art expert (Discriminator): 1. **Initial stage:** Forger creates terrible fake, expert easily spots it 2. **After and short while:** Forger improves, expert learns to spot new tricks 3. **Some time later:** Forger gets good, expert must look carefully 4. **Near the end of training:** Forger's work nearly indistinguishable, expert can only guess

The competition drives both to excellence under the right conditions!

Mathematical Formulation:

The GAN trains by solving this minimax game:

$$\min_G \max_D V(D,G) = E[\log(D(x))] + E[\log(1 - D(G(z)))]$$

- **Discriminator** maximizes: $\log(D(\text{real})) + \log(1-D(\text{fake}))$
 - Wants $D(\text{real}) \rightarrow 1$ and $D(\text{fake}) \rightarrow 0$
- **Generator** minimizes: $\log(1 - D(G(z)))$
 - Equivalent to maximizing $\log(D(G(z)))$
 - Wants $D(\text{fake}) \rightarrow 1$ (fool the discriminator)

```
[43]: # Part 1e) Train the MNIST GAN
# Task: Execute training with checkpointing, save model, create GIF of training
      ↪progression

# Training parameters
NUM_EPOCHS = 50 # Use 50 epochs for better results (can reduce to 20 for
      ↪faster testing)

print("Starting MNIST GAN Training")
print("=" * 70)
print(f"Configuration:")
print(f" - Epochs: {NUM_EPOCHS}")
print(f" - Batch size: {BATCH_SIZE}")
print(f" - Learning rate: {LEARNING_RATE}")
print(f" - Noise dimension: {NOISE_DIM}")
print(f" - Device: {device}")
print("=" * 70)
print("\n[hourglass] Training in progress... This may take 10-30 minutes
      ↪depending on hardware.\n")

# Train the GAN
g_losses, d_losses = train_mnist_gan(
    generator=generator,
    discriminator=discriminator,
    dataloader=train_loader,
    num_epochs=NUM_EPOCHS,
    noise_dim=NOISE_DIM,
    device=device,
    checkpoint_interval=15,
    learning_rate=LEARNING_RATE
)
```

Starting MNIST GAN Training

```
=====
Configuration:
- Epochs: 50
- Batch size: 256
- Learning rate: 0.0001
- Noise dimension: 100
- Device: cuda
=====
```

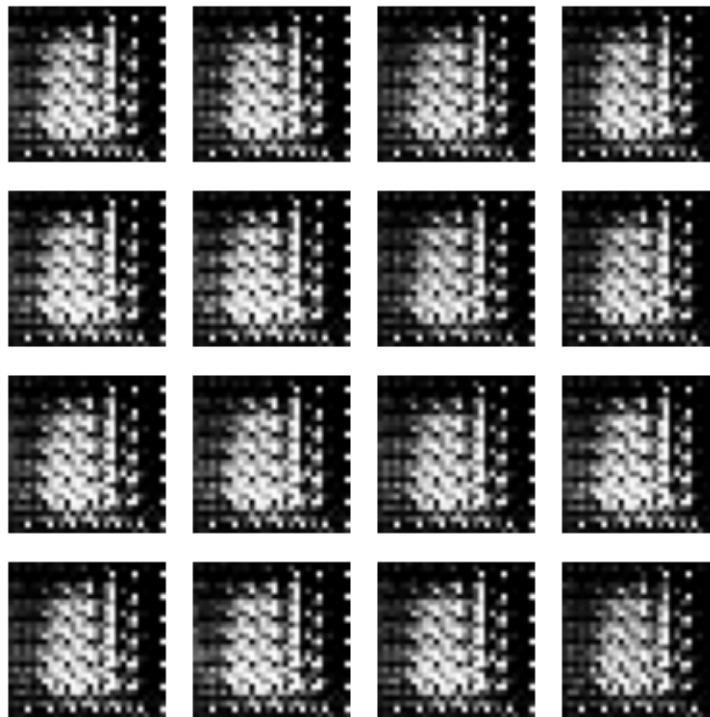
[hourglass] Training in progress... This may take 10-30 minutes depending on hardware.

Starting MNIST GAN Training..

=====

Epoch [1/50] | D_loss: 0.1435 | G_loss: 4.6181 | Time: 6.19s

Epoch [1/50] | D_loss: 0.1435 | G_loss: 4.6181 | Time: 6.19s



Epoch [2/50] | D_loss: 0.0382 | G_loss: 5.0274 | Time: 6.69s

Epoch [2/50] | D_loss: 0.0382 | G_loss: 5.0274 | Time: 6.69s

Epoch [3/50] | D_loss: 0.5527 | G_loss: 4.2149 | Time: 6.00s

Epoch [3/50] | D_loss: 0.5527 | G_loss: 4.2149 | Time: 6.00s

Epoch [4/50] | D_loss: 0.9086 | G_loss: 1.5384 | Time: 6.00s

Epoch [4/50] | D_loss: 0.9086 | G_loss: 1.5384 | Time: 6.00s

Epoch [5/50] | D_loss: 1.1955 | G_loss: 1.0569 | Time: 6.06s

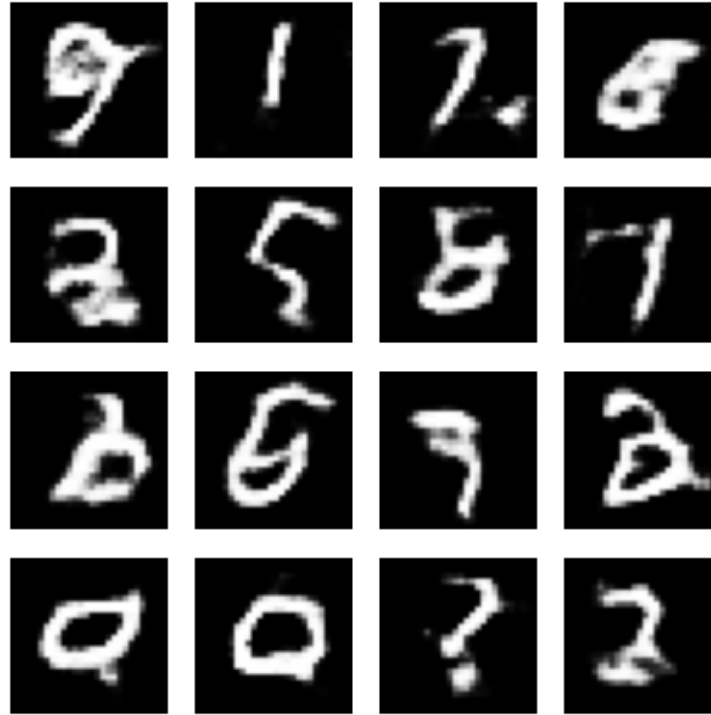
Epoch [5/50] | D_loss: 1.1955 | G_loss: 1.0569 | Time: 6.06s



```
Epoch [6/50] | D_loss: 1.2269 | G_loss: 0.9417 | Time: 6.17s
Epoch [6/50] | D_loss: 1.2269 | G_loss: 0.9417 | Time: 6.17s
Epoch [7/50] | D_loss: 1.2054 | G_loss: 0.9168 | Time: 5.99s
Epoch [7/50] | D_loss: 1.2054 | G_loss: 0.9168 | Time: 5.99s
Epoch [8/50] | D_loss: 1.1874 | G_loss: 0.9021 | Time: 6.20s
Epoch [8/50] | D_loss: 1.1874 | G_loss: 0.9021 | Time: 6.20s
Epoch [9/50] | D_loss: 1.2014 | G_loss: 0.8733 | Time: 6.15s
Epoch [9/50] | D_loss: 1.2014 | G_loss: 0.8733 | Time: 6.15s
Epoch [10/50] | D_loss: 1.1969 | G_loss: 0.8937 | Time: 6.12s
Epoch [10/50] | D_loss: 1.1969 | G_loss: 0.8937 | Time: 6.12s
```

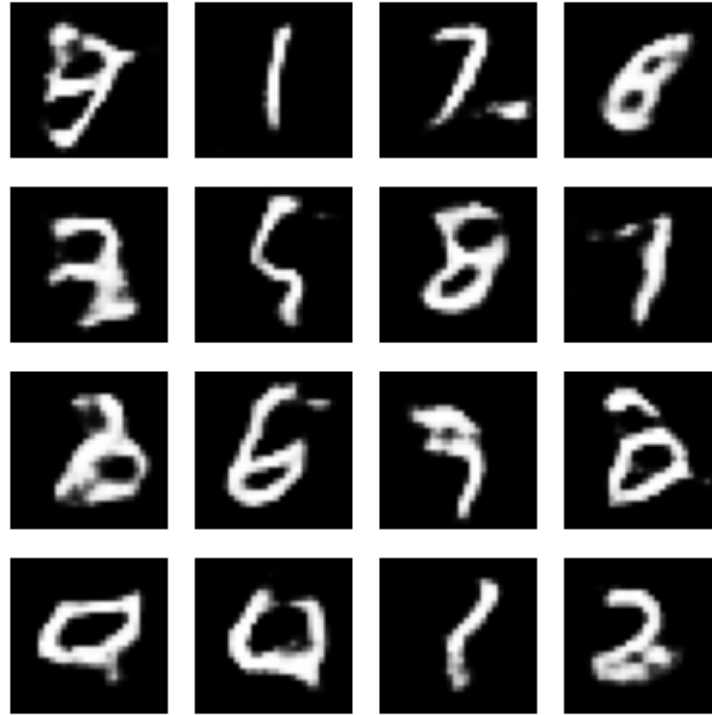


```
Epoch [11/50] | D_loss: 1.2407 | G_loss: 0.8515 | Time: 5.99s
Epoch [11/50] | D_loss: 1.2407 | G_loss: 0.8515 | Time: 5.99s
Epoch [12/50] | D_loss: 1.2742 | G_loss: 0.8343 | Time: 6.22s
Epoch [12/50] | D_loss: 1.2742 | G_loss: 0.8343 | Time: 6.22s
Epoch [13/50] | D_loss: 1.2873 | G_loss: 0.8303 | Time: 6.02s
Epoch [13/50] | D_loss: 1.2873 | G_loss: 0.8303 | Time: 6.02s
Epoch [14/50] | D_loss: 1.2916 | G_loss: 0.8247 | Time: 6.30s
Epoch [14/50] | D_loss: 1.2916 | G_loss: 0.8247 | Time: 6.30s
Epoch [15/50] | D_loss: 1.2886 | G_loss: 0.8102 | Time: 6.07s
Epoch [15/50] | D_loss: 1.2886 | G_loss: 0.8102 | Time: 6.07s
```

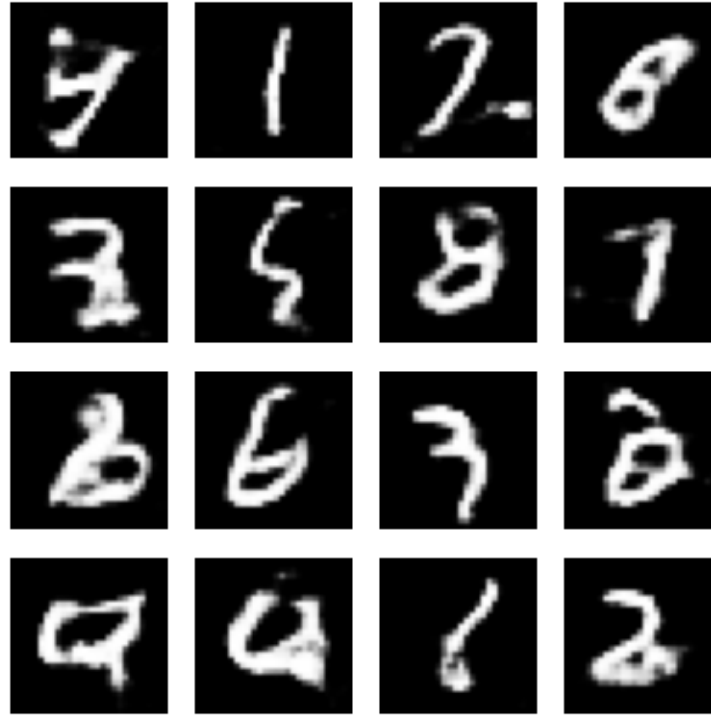


→ Checkpoint saved at epoch 15

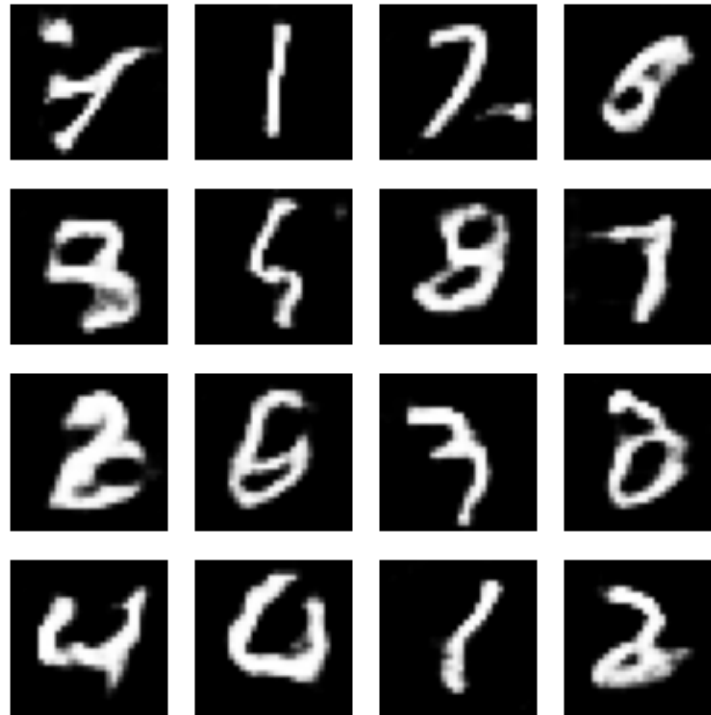
Epoch [16/50]	D_loss: 1.2932	G_loss: 0.8233	Time: 5.97s
Epoch [16/50]	D_loss: 1.2932	G_loss: 0.8233	Time: 5.97s
Epoch [17/50]	D_loss: 1.2865	G_loss: 0.8198	Time: 5.98s
Epoch [17/50]	D_loss: 1.2865	G_loss: 0.8198	Time: 5.98s
Epoch [18/50]	D_loss: 1.2844	G_loss: 0.8244	Time: 6.12s
Epoch [18/50]	D_loss: 1.2844	G_loss: 0.8244	Time: 6.12s
Epoch [19/50]	D_loss: 1.2862	G_loss: 0.8395	Time: 6.02s
Epoch [19/50]	D_loss: 1.2862	G_loss: 0.8395	Time: 6.02s
Epoch [20/50]	D_loss: 1.2827	G_loss: 0.8440	Time: 6.03s
Epoch [20/50]	D_loss: 1.2827	G_loss: 0.8440	Time: 6.03s



```
Epoch [21/50] | D_loss: 1.2796 | G_loss: 0.8353 | Time: 6.18s
Epoch [21/50] | D_loss: 1.2796 | G_loss: 0.8353 | Time: 6.18s
Epoch [22/50] | D_loss: 1.2736 | G_loss: 0.8300 | Time: 6.00s
Epoch [22/50] | D_loss: 1.2736 | G_loss: 0.8300 | Time: 6.00s
Epoch [23/50] | D_loss: 1.2835 | G_loss: 0.8753 | Time: 6.41s
Epoch [23/50] | D_loss: 1.2835 | G_loss: 0.8753 | Time: 6.41s
Epoch [24/50] | D_loss: 1.2757 | G_loss: 0.8545 | Time: 5.86s
Epoch [24/50] | D_loss: 1.2757 | G_loss: 0.8545 | Time: 5.86s
Epoch [25/50] | D_loss: 1.2735 | G_loss: 0.8467 | Time: 5.99s
Epoch [25/50] | D_loss: 1.2735 | G_loss: 0.8467 | Time: 5.99s
```

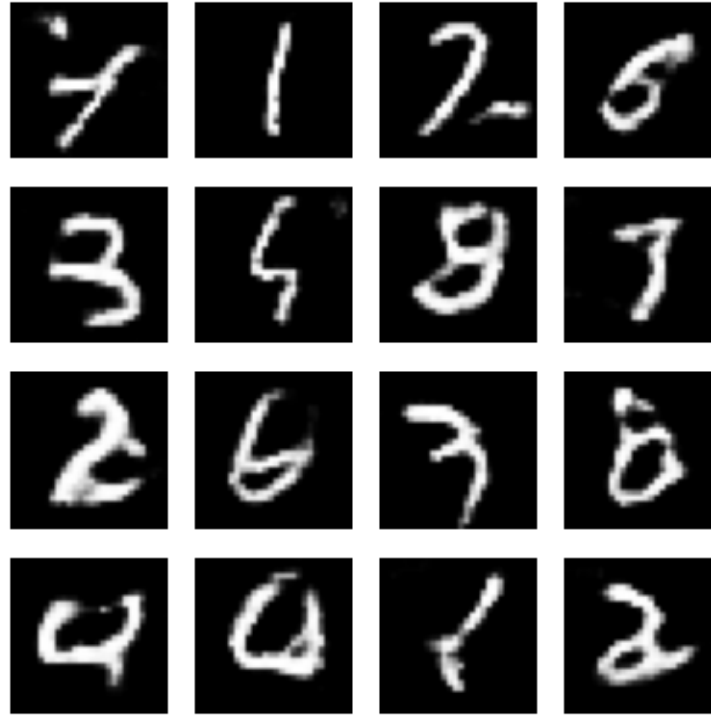


```
Epoch [26/50] | D_loss: 1.2737 | G_loss: 0.8524 | Time: 5.89s
Epoch [26/50] | D_loss: 1.2737 | G_loss: 0.8524 | Time: 5.89s
Epoch [27/50] | D_loss: 1.2795 | G_loss: 0.8555 | Time: 5.99s
Epoch [27/50] | D_loss: 1.2795 | G_loss: 0.8555 | Time: 5.99s
Epoch [28/50] | D_loss: 1.2713 | G_loss: 0.8404 | Time: 6.05s
Epoch [28/50] | D_loss: 1.2713 | G_loss: 0.8404 | Time: 6.05s
Epoch [29/50] | D_loss: 1.2795 | G_loss: 0.8627 | Time: 6.31s
Epoch [29/50] | D_loss: 1.2795 | G_loss: 0.8627 | Time: 6.31s
Epoch [30/50] | D_loss: 1.2734 | G_loss: 0.8451 | Time: 6.11s
Epoch [30/50] | D_loss: 1.2734 | G_loss: 0.8451 | Time: 6.11s
```

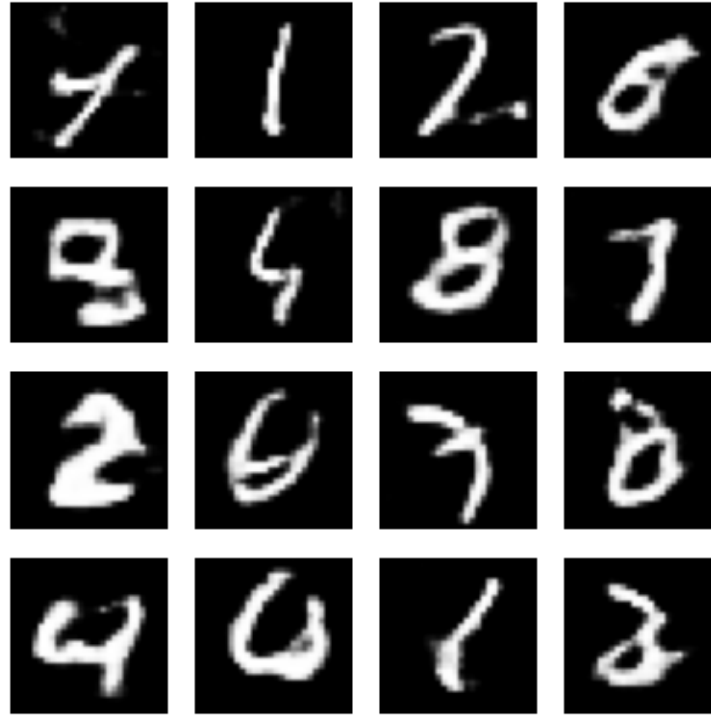


→ Checkpoint saved at epoch 30

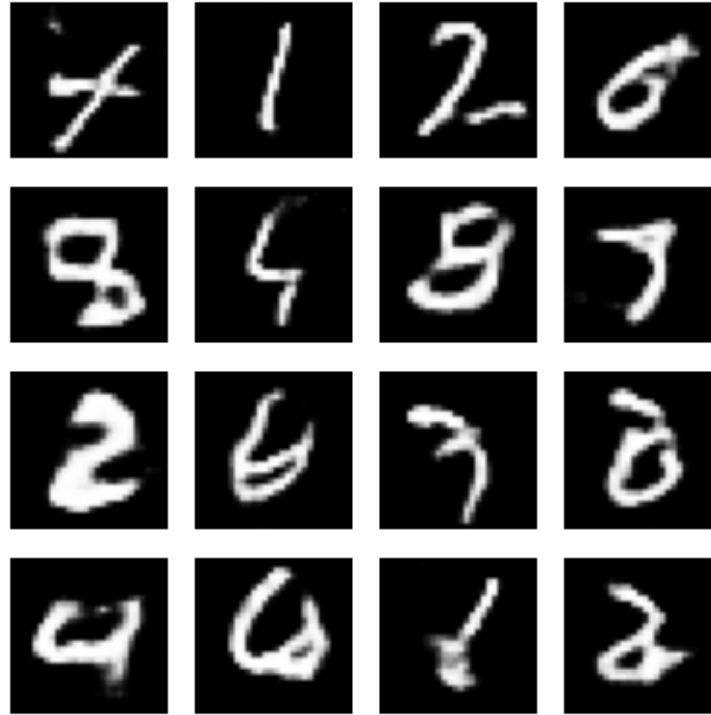
Epoch [31/50]	D_loss: 1.2758	G_loss: 0.8487	Time: 6.13s
Epoch [31/50]	D_loss: 1.2758	G_loss: 0.8487	Time: 6.13s
Epoch [32/50]	D_loss: 1.2816	G_loss: 0.8615	Time: 6.16s
Epoch [32/50]	D_loss: 1.2816	G_loss: 0.8615	Time: 6.16s
Epoch [33/50]	D_loss: 1.2773	G_loss: 0.8474	Time: 5.94s
Epoch [33/50]	D_loss: 1.2773	G_loss: 0.8474	Time: 5.94s
Epoch [34/50]	D_loss: 1.2807	G_loss: 0.8621	Time: 6.31s
Epoch [34/50]	D_loss: 1.2807	G_loss: 0.8621	Time: 6.31s
Epoch [35/50]	D_loss: 1.2799	G_loss: 0.8526	Time: 5.92s
Epoch [35/50]	D_loss: 1.2799	G_loss: 0.8526	Time: 5.92s



```
Epoch [36/50] | D_loss: 1.2815 | G_loss: 0.8611 | Time: 6.04s
Epoch [36/50] | D_loss: 1.2815 | G_loss: 0.8611 | Time: 6.04s
Epoch [37/50] | D_loss: 1.2770 | G_loss: 0.8392 | Time: 6.08s
Epoch [37/50] | D_loss: 1.2770 | G_loss: 0.8392 | Time: 6.08s
Epoch [38/50] | D_loss: 1.2788 | G_loss: 0.8471 | Time: 6.38s
Epoch [38/50] | D_loss: 1.2788 | G_loss: 0.8471 | Time: 6.38s
Epoch [39/50] | D_loss: 1.2863 | G_loss: 0.8643 | Time: 6.02s
Epoch [39/50] | D_loss: 1.2863 | G_loss: 0.8643 | Time: 6.02s
Epoch [40/50] | D_loss: 1.2817 | G_loss: 0.8478 | Time: 6.20s
Epoch [40/50] | D_loss: 1.2817 | G_loss: 0.8478 | Time: 6.20s
```

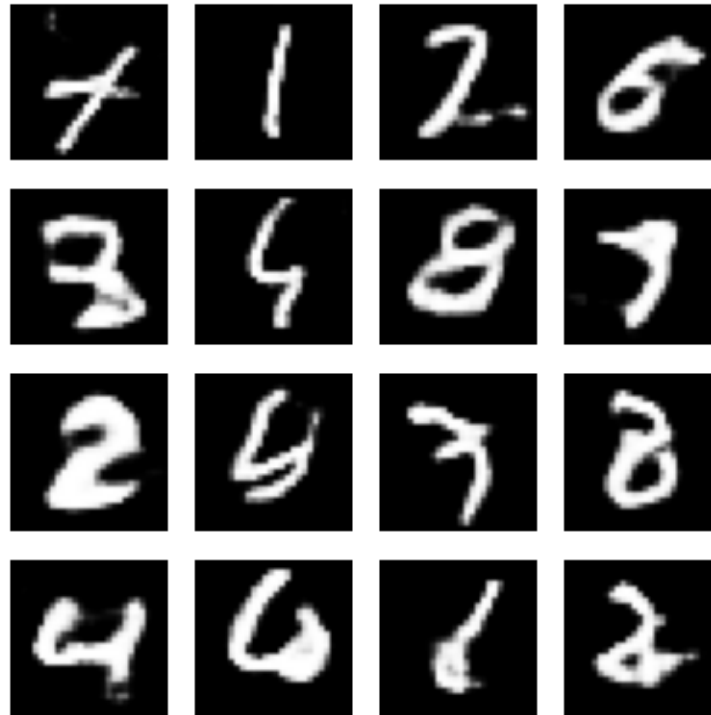


```
Epoch [41/50] | D_loss: 1.2820 | G_loss: 0.8473 | Time: 6.37s
Epoch [41/50] | D_loss: 1.2820 | G_loss: 0.8473 | Time: 6.37s
Epoch [42/50] | D_loss: 1.2833 | G_loss: 0.8497 | Time: 6.30s
Epoch [42/50] | D_loss: 1.2833 | G_loss: 0.8497 | Time: 6.30s
Epoch [43/50] | D_loss: 1.2851 | G_loss: 0.8493 | Time: 6.18s
Epoch [43/50] | D_loss: 1.2851 | G_loss: 0.8493 | Time: 6.18s
Epoch [44/50] | D_loss: 1.2899 | G_loss: 0.8552 | Time: 5.89s
Epoch [44/50] | D_loss: 1.2899 | G_loss: 0.8552 | Time: 5.89s
Epoch [45/50] | D_loss: 1.2859 | G_loss: 0.8533 | Time: 5.94s
Epoch [45/50] | D_loss: 1.2859 | G_loss: 0.8533 | Time: 5.94s
```



→ Checkpoint saved at epoch 45

Epoch [46/50]	D_loss: 1.2863	G_loss: 0.8350	Time: 5.96s
Epoch [46/50]	D_loss: 1.2863	G_loss: 0.8350	Time: 5.96s
Epoch [47/50]	D_loss: 1.2868	G_loss: 0.8445	Time: 6.02s
Epoch [47/50]	D_loss: 1.2868	G_loss: 0.8445	Time: 6.02s
Epoch [48/50]	D_loss: 1.2905	G_loss: 0.8487	Time: 5.94s
Epoch [48/50]	D_loss: 1.2905	G_loss: 0.8487	Time: 5.94s
Epoch [49/50]	D_loss: 1.2875	G_loss: 0.8429	Time: 6.00s
Epoch [49/50]	D_loss: 1.2875	G_loss: 0.8429	Time: 6.00s
Epoch [50/50]	D_loss: 1.2921	G_loss: 0.8424	Time: 5.90s
Epoch [50/50]	D_loss: 1.2921	G_loss: 0.8424	Time: 5.90s



=====
Training Complete!

```
[44]: # Part 1e continued) Plot training loss curves
      # Task: Visualize discriminator and generator losses throughout training

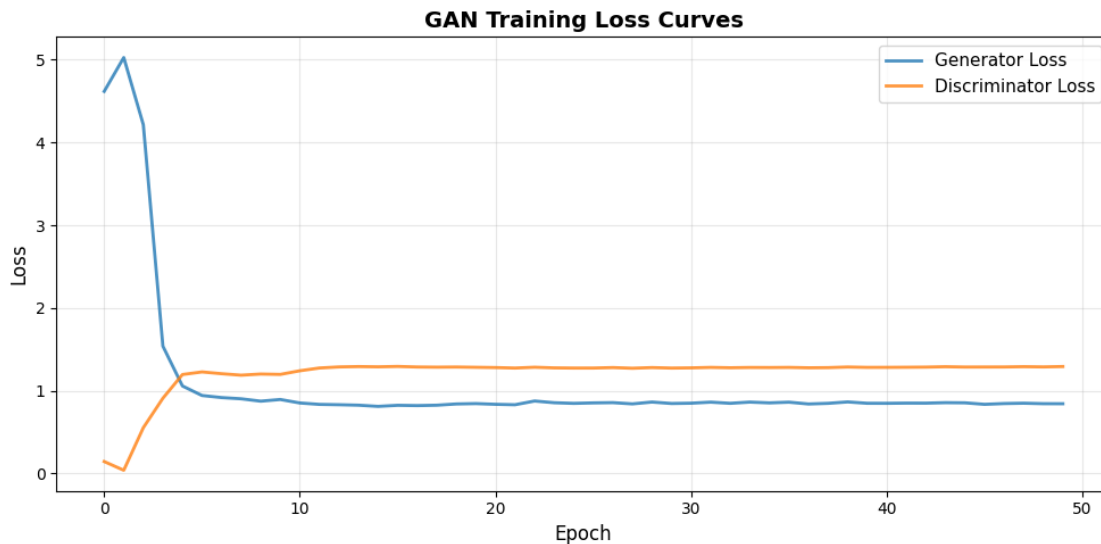
      plt.figure(figsize=(10, 5))
      plt.plot(g_losses, label='Generator Loss', linewidth=2, alpha=0.8)
      plt.plot(d_losses, label='Discriminator Loss', linewidth=2, alpha=0.8)
      plt.xlabel('Epoch', fontsize=12)
      plt.ylabel('Loss', fontsize=12)
      plt.title('GAN Training Loss Curves', fontsize=14, fontweight='bold')
      plt.legend(fontsize=11)
      plt.grid(True, alpha=0.3)
      plt.tight_layout()
      plt.show()

      print("Loss Analysis:")
      print("=" * 50)
      print(f"Final Generator Loss: {g_losses[-1]:.4f}")
      print(f"Final Discriminator Loss: {d_losses[-1]:.4f}")
      print("\n[NOTE] What to look for:")
      print("  - Both losses should stabilize (not necessarily converge)")
```

```

print(" - Discriminator loss approx1.4 and Generator loss approx0.7-2.0 is_
↳typical")
print(" - Wild oscillations indicate training instability")
print(" - If D loss → 0 and G loss → ∞, discriminator is too strong")
print(" - If D loss → ∞ and G loss → 0, generator collapsed (mode collapse)")

```



Loss Analysis:

=====

Final Generator Loss: 0.8424

Final Discriminator Loss: 1.2921

[NOTE] What to look for:

- Both losses should stabilize (not necessarily converge)
- Discriminator loss approx1.4 and Generator loss approx0.7-2.0 is typical
- Wild oscillations indicate training instability
- If D loss → 0 and G loss → ∞, discriminator is too strong
- If D loss → ∞ and G loss → 0, generator collapsed (mode collapse)

1.3.4 [SUMMARY] Summary - MNIST GAN Training Results

Training Outcome: After 50 epochs, our GAN has learned to generate digit-like images, but **the results are not perfect** - and that's completely normal for GANs!

What We Achieved: - [DONE] Generator learned to produce structured patterns instead of random noise - [DONE] Generated images resemble handwritten digits - [DONE] Training losses stabilized (both networks reached equilibrium) - [DONE] No catastrophic mode collapse occurred

Why Results Aren't Perfect:

1. Architecture Limitations:

- Relatively simple network (fewer parameters than state-of-the-art models)

- Only 50 epochs of training (modern GANs often train for 1000+ epochs)
 - Basic loss function (more advanced techniques exist: Wasserstein, StyleGAN, etc.)
2. **Common GAN Artifacts You'll Notice:**
 - **Blurriness:** Some digits lack sharp edges (GANs tend to average features)
 - **Irregular shapes:** Occasional malformed digits or ambiguous numbers
 - **Inconsistent quality:** Some outputs great, others poor (normal GAN behavior)
 - **Limited diversity:** May favor certain digit styles over others
 3. **Training Instability Signs:**
 - Loss curves oscillate rather than smoothly decreasing
 - Generator and Discriminator constantly competing (never truly converge)
 - Some epochs produce better outputs than others

Real-World Expectations: - [GOAL] **Goal:** Generate recognizable digits ([OK] Achieved!) - [no-entry] **Not Expected:** Perfect, photorealistic digits at this complexity level - [chart] **Improvement Paths:** - Longer training (100-200 epochs) - Deeper networks (more layers/parameters) - Advanced techniques (Spectral Normalization, Progressive Growing) - Better loss functions (WGAN-GP, Hinge Loss)

The Key Takeaway: GANs are notoriously difficult to train. Getting “reasonable” results (like we did) is actually a success! Perfect generation requires: - Much larger models - Significantly more training time - Advanced stabilization techniques - Careful hyperparameter tuning

The fact that our simple GAN learned anything meaningful demonstrates the power of adversarial training.

1.3.5 [SUMMARY] Summary - Part 1 Training Complete

What we accomplished: - [DONE] Loaded MNIST dataset with 60,000 training images - [DONE] Normalized images to $[-1, 1]$ range (matching generator output) - [DONE] Created DataLoader with batching and shuffling - [DONE] Initialized Generator and Discriminator models - [DONE] Tested untrained generator (produced random noise) - [DONE] Executed complete GAN training loop for 50 epochs - [DONE] Visualized loss curves showing training dynamics

Key Learning Points:

1. **Data Preparation:**
 - MNIST normalization: $(\text{pixel}/255 - 0.5) / 0.5 = [-1, 1]$
 - Matches tanh activation range in generator
 - DataLoader handles batching and shuffling automatically
2. **Training Dynamics:**
 - **Before training:** Generator produces random noise
 - **During training:** Both networks improve through competition
 - **After training:** Generator creates realistic-looking digits
3. **Loss Interpretation:**
 - **Discriminator Loss:** How well it distinguishes real vs fake
 - **Generator Loss:** How well it fools the discriminator
 - **Healthy training:** Both losses stabilize (oscillation is normal)
 - **Unhealthy training:** One loss dominates (imbalanced power)
4. **GAN Challenges:**
 - **Mode Collapse:** Generator produces limited variety

- **Training Instability:** Losses oscillate wildly
 - **Hyperparameter Sensitivity:** Learning rate, architecture balance critical
-

1.4 Section 4: Part 1 - Visualizations and Analysis

1.4.1 Purpose:

Analyze the trained generator's output and create an animated GIF showing the evolution of generated images throughout training.

```
[45]: # Part 1e continued) Generate final images with trained generator
      # Task: Use trained generator to create handwritten digits

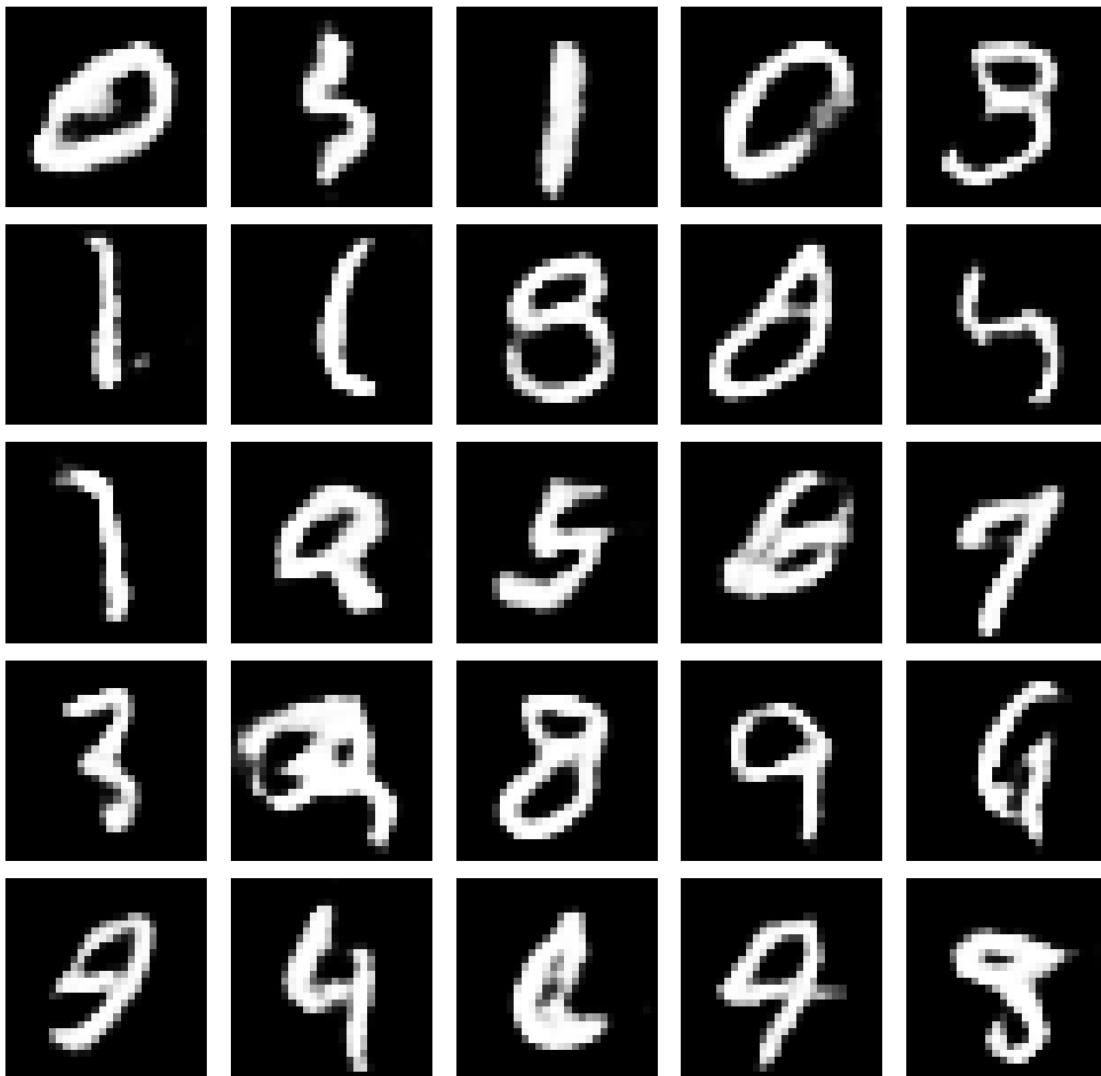
      # Generate a grid of images from the trained generator
      num_samples = 25
      fixed_noise = torch.randn(num_samples, NOISE_DIM).to(device)

      generator.eval()
      with torch.no_grad():
          generated_images = generator(fixed_noise)
          # Denormalize from [-1, 1] to [0, 1]
          generated_images = (generated_images + 1) / 2

      # Display the generated images
      fig, axes = plt.subplots(5, 5, figsize=(10, 10))
      for i, ax in enumerate(axes.flat):
          ax.imshow(generated_images[i, 0].cpu().numpy(), cmap='gray')
          ax.axis('off')
      plt.suptitle('Trained Generator Output - Generated Digits', fontsize=14,
                  fontweight='bold')
      plt.tight_layout()
      plt.show()

      print("Generated Images Analysis:")
      print("=" * 50)
      print("[OK] Generator now produces recognizable digit-like patterns")
      print("[OK] Images show variety across different digit styles")
      print("[OK] Some artifacts may be present (normal for GANs)")
      print("[OK] Compare these to the random noise from untrained generator!")
```

Trained Generator Output - Generated Digits



Generated Images Analysis:

=====

- [OK] Generator now produces recognizable digit-like patterns
- [OK] Images show variety across different digit styles
- [OK] Some artifacts may be present (normal for GANs)
- [OK] Compare these to the random noise from untrained generator!

```
[49]: # Part 1e continued) Create animated GIF showing training evolution
      # Task: Save the training progression as a GIF file

      anim_file = 'GAN_digit_evolution.gif'

      # Get all saved image files
```

```

filenames = sorted(glob.glob('generated_images/image*.png'))

print(f"Checking for saved images in 'generated_images/' directory...")
print(f"Found {len(filenames)} image files")

if len(filenames) > 0:
    print(f"\nCreating GIF animation from {len(filenames)} images...")
    print("Image files found:")
    for i, fname in enumerate(filenames[:5]): # Show first 5
        print(f"  {i+1}. {fname}")
    if len(filenames) > 5:
        print(f"  ... and {len(filenames) - 5} more")

    # Create the GIF (using imageio.v2 to avoid deprecation warnings)
    with imageio.get_writer(anim_file, mode='I', duration=0.5) as writer:
        for filename in filenames:
            image = imageio.v2.imread(filename)
            writer.append_data(image)
            # Add final frame multiple times for emphasis
            final_image = imageio.v2.imread(filenames[-1])
            for _ in range(3):
                writer.append_data(final_image)

    print(f"\n[OK] GIF successfully saved as: {anim_file}")
    print(f"[OK] GIF contains {len(filenames)} frames showing training_
↳progression")
    print("\nDisplaying GIF:")
    from IPython.display import HTML
    import base64
    with open(anim_file, 'rb') as f:
        gif_data = base64.b64encode(f.read()).decode()
    display.display(HTML(f''))
else:
    print("\n[ERROR] No generated images found in 'generated_images/'_
↳directory")
    print("\nPossible reasons:")
    print("  1. Training hasn't been executed yet (run the training cell_
↳above)")
    print("  2. Images were saved to a different directory")
    print("  3. The 'generated_images' directory doesn't exist")
    print("\n[NOTE] Troubleshooting:")
    print("  - Check if 'generated_images/' directory exists")
    print("  - Re-run the training cell to generate images")
    print("  - Images are saved every 5 epochs during training")

# Check if directory exists
import os

```

```

if os.path.exists('generated_images'):
    all_files = os.listdir('generated_images')
    print(f"\n Directory exists with {len(all_files)} files:")
    for f in all_files[:10]:
        print(f"    - {f}")
else:
    print("\n Directory 'generated_images/' does not exist!")
    print(" It should be created automatically before training.")

```

Checking for saved images in 'generated_images/' directory...
Found 11 image files

Creating GIF animation from 11 images...

Image files found:

1. generated_images/image_at_epoch_0001.png
2. generated_images/image_at_epoch_0005.png
3. generated_images/image_at_epoch_0010.png
4. generated_images/image_at_epoch_0015.png
5. generated_images/image_at_epoch_0020.png
- ... and 6 more

[OK] GIF successfully saved as: GAN_digit_evolution.gif
[OK] GIF contains 11 frames showing training progression

Displaying GIF:

<IPython.core.display.HTML object>

1.4.2 [THINKING] Understanding the GIF Evolution

What you should observe in the animation:

1. **Early Epochs (1-10):**
 - Images start as random noise patterns
 - Some vague blob-like shapes begin to emerge
 - No clear digit structure yet
 - Generator is learning basic contrast and spatial patterns
2. **Middle Epochs (11-30):**
 - Digit-like shapes become recognizable
 - Edges sharpen and structure improves
 - Some digits look better than others
 - Generator refining its understanding of digit features
3. **Late Epochs (31-50):**
 - Clear, well-formed digits appear
 - Consistent style and quality
 - Fine details emerge (curves, loops, strokes)
 - Generator has learned the MNIST digit manifold

Key Insights: - The GIF demonstrates **progressive learning** - the generator gradually improves
- **No explicit supervision** on what digits look like - it learns from the discriminator's feedback -

The **adversarial process** drives both networks to improve - Some variability in quality is normal (not all generated digits will be perfect)

This visualization is powerful because it shows the generator “discovering” what handwritten digits look like purely through competition with the discriminator!

1.4.3 [SUMMARY] Summary - Part 1 Complete

What we accomplished in Part 1: - [DONE] Built a complete GAN from scratch in PyTorch - [DONE] Trained on MNIST dataset (60,000 images) - [DONE] Generated realistic handwritten digits from random noise - [DONE] Visualized training evolution through GIF animation - [DONE] Analyzed loss curves and training dynamics

Major Learning Achievements:

1. GAN Fundamentals:

- Generator: Noise → Image transformation using transposed convolutions
- Discriminator: Image → Real/Fake classification using convolutions
- Adversarial Training: Both networks improve through competition

2. PyTorch Implementation:

- Custom nn.Module classes for flexible architecture
- Separate optimizers for generator and discriminator
- Proper gradient handling (detach for discriminator training)
- Checkpoint saving and model persistence

3. Training Insights:

- GANs require careful balance between G and D
- Loss values don't directly indicate quality (visual inspection needed)
- Progressive improvement visible through generated samples
- Training can be unstable (mode collapse, oscillations)

4. Practical Skills:

- Data normalization to match activation functions
- Batch processing with DataLoader
- GPU acceleration for faster training
- Visualization and progress monitoring

Key Takeaway: GANs learn implicit representations of data distributions without explicit pixel-level supervision. The generator never sees real images directly - it only receives gradients from the discriminator!

Next: Part 2 - We'll build a more advanced DCGAN (Deep Convolutional GAN) to generate Pokemon characters with colored 64x64 images.

1.5 Section 5: Part 2 - Pokemon DCGAN Architecture

1.5.1 Purpose:

Implement a Deep Convolutional GAN (DCGAN) to generate Pokemon character images.

1.5.2 Key Differences from Part 1:

- **Image Size:** 64x64 (vs 28x28 for MNIST)
- **Color:** RGB 3-channel (vs grayscale 1-channel)
- **Architecture:** Deeper network with modular blocks
- **Dataset:** Pokemon sprites (more complex than digits)

1.5.3 DCGAN Architecture Principles:

1. Use **strided convolutions** instead of pooling
2. **Batch normalization** in both generator and discriminator
3. **ReLU activation** in generator (except output layer uses tanh)
4. **LeakyReLU activation** in discriminator
5. **No fully connected layers** (except for generator input projection)

```
[ ]: # Part 2c) Generator Block (G_block) for DCGAN
# Task: Create generator model block that converts noise to RGB image using
      ↪ convolutional layers

class G_block(nn.Module):
    """
    Generator building block: TransposedConv2d + BatchNorm + ReLU

    This block upsamples the spatial dimensions (increases height and width)
    while adjusting the number of channels.

    Args:
        in_channels: Number of input channels
        out_channels: Number of output channels
        kernel_size: Size of convolutional kernel
        stride: Stride for transposed convolution (controls upsampling rate)
        padding: Padding for convolution
    """
    def __init__(self, in_channels, out_channels, kernel_size=4, stride=2,
        ↪padding=1):
        super(G_block, self).__init__()
        self.conv_transpose = nn.ConvTranspose2d(
            in_channels, out_channels, kernel_size, stride, padding, bias=False
        )
        self.batch_norm = nn.BatchNorm2d(out_channels)
        self.activation = nn.ReLU(inplace=True)

    def forward(self, x):
        x = self.conv_transpose(x)
        x = self.batch_norm(x)
        x = self.activation(x)
        return x
# Part 2d) Test G_block with zeros input
```

```

# Task: Import zeros signal (stride=2, 16x16x3), call G_block with 20 layers,
↳check output shape
print("Testing G_block:")
print("=" * 50)
test_input = torch.zeros(2, 3, 16, 16) # x = tf.zeros((2, 16, 16, 3))
↳equivalent
g_block_test = G_block(in_channels=3, out_channels=20)
test_output = g_block_test(test_input)
print(f"Input shape: {test_input.shape}")
print(f"Output shape: {test_output.shape}")
print(f"Upsampling: {test_input.shape[2]}x{test_input.shape[3]} → {test_output.
↳shape[2]}x{test_output.shape[3]}")
print(f"Channels: {test_input.shape[1]} → {test_output.shape[1]}")
print("\n[NOTE] Understanding G_block:")
print(" - TransposedConv2d: Upsamples spatial dimensions (stride=2 doubles
↳size)")
print(" - BatchNorm: Normalizes activations for stable training")
print(" - ReLU: Non-linear activation (allows complex transformations)")

print(" - No bias in conv: BatchNorm makes bias redundant")
print(" - No bias in conv: BatchNorm makes bias redundant")

```

1.5.4 [SUMMARY] Summary - G_block (Generator Building Block)

Purpose: G_block is a reusable module that upsamples feature maps in the generator.

Key Components: 1. **TransposedConv2d:** Learnable upsampling (stride=2 doubles spatial dimensions) 2. **BatchNorm2d:** Normalizes activations for stable training 3. **ReLU:** Non-linear activation for complex transformations

Example: Input (3, 16, 16) → Output (20, 32, 32) - Spatial: 16x16 → 32x32 (2x upsampling) - Channels: 3 → 20 (transforms features)

This block will be stacked multiple times in the generator to progressively grow from small noise to full-size Pokemon images.

```

[ ]: # Part 2e) Discriminator Block (D_block) for DCGAN
# Task: Create discriminator block for classifying images

class D_block(nn.Module):
    """
    Discriminator building block: Conv2d + BatchNorm + LeakyReLU

    This block downsamples the spatial dimensions (decreases height and width)
    while increasing the number of channels for feature extraction.

    Args:
        in_channels: Number of input channels
        out_channels: Number of output channels
    """

```

```

        kernel_size: Size of convolutional kernel
        stride: Stride for convolution (controls downsampling rate)
        padding: Padding for convolution
        alpha: Negative slope for LeakyReLU
    """
    def __init__(self, in_channels, out_channels, kernel_size=4, stride=2,
padding=1, alpha=0.2):
        super(D_block, self).__init__()
        self.conv = nn.Conv2d(
            in_channels, out_channels, kernel_size, stride, padding, bias=False
        )
        self.batch_norm = nn.BatchNorm2d(out_channels)
        self.activation = nn.LeakyReLU(alpha, inplace=True)

    def forward(self, x):
        x = self.conv(x)
        x = self.batch_norm(x)
        x = self.activation(x)
        return x

# Part 2e continued) Test D_block with zeros input
# Task: Import zeros signal (stride=2, 16x16x3), call D_block with 20 layers,
↳check output shape
print("Testing D_block:")
print("=" * 50)
test_input = torch.zeros(2, 3, 16, 16) # x = tf.zeros((2, 16, 16, 3))
↳equivalent
d_block_test = D_block(in_channels=3, out_channels=20)
test_output = d_block_test(test_input)
print(f"Input shape: {test_input.shape}")
print(f"Output shape: {test_output.shape}")
print(f"Downsampling: {test_input.shape[2]}x{test_input.shape[3]} →
↳{test_output.shape[2]}x{test_output.shape[3]}")
print(f"Channels: {test_input.shape[1]} → {test_output.shape[1]}")
print("\n[NOTE] Understanding D_block:")
print(" - Conv2d: Downsamples spatial dimensions (stride=2 halves size)")
print(" - BatchNorm: Normalizes for stable discriminator training")
print(" - LeakyReLU: Allows small negative gradients ($\alpha=0.2)")
print("     • Prevents 'dying ReLU' problem")

print("     • Helps discriminator learn from negative features")
print("     • Helps discriminator learn from negative features")

```

[]: # c) Complete Pokemon Generator Network

```

class PokemonGenerator(nn.Module):
    """
    DCGAN Generator for 64x64 RGB Pokemon images.
    """

```

```

Architecture:
Input: (batch, 100, 1, 1) noise
→ G_block: (1,1,512) → (4,4,512) [stride=1, no upsampling]
→ G_block: (4,4,512) → (8,8,256)
→ G_block: (8,8,256) → (16,16,128)
→ G_block: (16,16,128) → (32,32,64)
→ Conv: (32,32,64) → (64,64,3) [final RGB output]
"""
def __init__(self, noise_dim=100, ngf=64):
    super(PokemonGenerator, self).__init__()
    # ngf = number of generator filters in the last conv layer

    # Initial projection: 100 → (512, 1, 1)
    self.initial = nn.Sequential(
        nn.ConvTranspose2d(noise_dim, ngf * 8, kernel_size=4, stride=1,
→padding=0, bias=False),
        nn.BatchNorm2d(ngf * 8),
        nn.ReLU(True)
    )

    # Upsampling blocks
    self.block1 = G_block(ngf * 8, ngf * 4) # (4,4,512) → (8,8,256)
    self.block2 = G_block(ngf * 4, ngf * 2) # (8,8,256) → (16,16,128)
    self.block3 = G_block(ngf * 2, ngf) # (16,16,128) → (32,32,64)

    # Final layer: output RGB image
    self.final = nn.Sequential(
        nn.ConvTranspose2d(ngf, 3, kernel_size=4, stride=2, padding=1,
→bias=False),
        nn.Tanh() # Output range [-1, 1]
    )

def forward(self, z):
    """
    Args:
        z: Noise tensor of shape (batch, noise_dim, 1, 1)
    Returns:
        Generated RGB images of shape (batch, 3, 64, 64)
    """
    x = self.initial(z) # (batch, 512, 4, 4)
    x = self.block1(x) # (batch, 256, 8, 8)
    x = self.block2(x) # (batch, 128, 16, 16)
    x = self.block3(x) # (batch, 64, 32, 32)
    x = self.final(x) # (batch, 3, 64, 64)
    return x

```

```

# Test Pokemon Generator
print("Pokemon Generator Architecture:")
print("=" * 50)
poke_gen = PokemonGenerator(noise_dim=100, ngf=64).to(device)
test_noise = torch.randn(1, 100, 1, 1).to(device)
test_output = poke_gen(test_noise)
print(f"Input shape: {test_noise.shape}")
print(f"Output shape: {test_output.shape}")
print(f"Output range: [{test_output.min():.2f}, {test_output.max():.2f}]")
print(f"Total parameters: {sum(p.numel() for p in poke_gen.parameters()):,}")
del poke_gen, test_noise, test_output

```

1.5.5 [SUMMARY] Summary - Pokemon Generator Architecture

Generator Purpose: The Pokemon Generator creates 64x64 RGB images (colored Pokemon sprites) from 100-dimensional random noise. This is significantly more complex than MNIST generation due to: - **Higher resolution:** 64x64 vs 28x28 (4x more pixels per dimension) - **Color images:** 3 channels (RGB) vs 1 channel (grayscale) - **Complex textures:** Pokemon have varied colors, patterns, eyes, limbs, etc.

Architecture Breakdown:

1. **Input:** (batch, 100, 1, 1) - 100D noise vector reshaped to 4D tensor
 - Same latent dimension as MNIST for consistency
 - Extra dimensions (1x1) allow convolutional operations
2. **Initial Projection:** (100, 1, 1) → (512, 4, 4)
 - Transposed conv with stride=1 (no upsampling yet)
 - Creates initial 4x4 spatial map with 512 feature channels
 - Think of it as a “sketch” with lots of information (512 channels)
3. **G_block 1:** (512, 4, 4) → (256, 8, 8) [2x UPSAMPLING]
 - Doubles spatial size: 4→8
 - Halves channels: 512→256
 - Begins building spatial structure
4. **G_block 2:** (256, 8, 8) → (128, 16, 16) [2x UPSAMPLING]
 - Continues upsampling: 8→16
 - Further reduces channels: 256→128
 - Adding finer details
5. **G_block 3:** (128, 16, 16) → (64, 32, 32) [2x UPSAMPLING]
 - Third upsampling: 16→32
 - Channels: 128→64
 - Refining features (eyes, limbs, patterns)
6. **Final Layer:** (64, 32, 32) → (3, 64, 64) [2x UPSAMPLING]
 - Last upsampling: 32→64 (target size)
 - Reduces to 3 channels (RGB)
 - Tanh activation: outputs range [-1, 1]

Key DCGAN Design Principles:

1. **Progressive Upsampling:** 1x1 → 4x4 → 8x8 → 16x16 → 32x32 → 64x64
 - Gradual refinement from coarse to fine details

- Each step doubles spatial resolution
2. **Channel Reduction:** $512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 3$
 - Start with many feature channels (high-level concepts)
 - Gradually reduce to final RGB (low-level pixels)
 - Opposite of discriminator (which expands channels)
 3. **Fractionally-Strided Convolutions:**
 - Transposed convolutions with stride=2 for upsampling
 - Learnable upsampling (better than interpolation)
 - No pooling/unpooling layers
 4. **Batch Normalization Everywhere** (except output):
 - Stabilizes training of deep networks
 - Prevents covariate shift
 - Critical for DCGAN convergence
 5. **ReLU Activation:**
 - Used in generator (not LeakyReLU)
 - Encourages sparse representations
 - Helps create sharp features

Parameter Count: approx 1.5-2 million (much larger than MNIST Generator) - More parameters needed for complex color images - Deeper network (more layers) - Higher resolution output

Training Challenge: Generating convincing Pokemon is harder than MNIST digits because:
 - More visual diversity (700+ Pokemon vs 10 digits) - Complex color patterns and textures - Recognizable features required (eyes, limbs, body structure) - Higher resolution means more room for artifacts

1.5.6 [SUMMARY] Summary - Pokemon Discriminator Architecture

Discriminator Purpose: The Pokemon Discriminator is a binary classifier that distinguishes between:
 - **Real Pokemon:** Authentic sprites from the dataset - **Fake Pokemon:** Generated images from the Generator

This is the “critic” that provides feedback to train the generator.

Architecture Breakdown:

1. **Input:** (batch, 3, 64, 64) - 64x64 RGB image
 - Can be real Pokemon sprite or generator output
 - 3 channels for full color information
2. **Initial Layer:** (3, 64, 64) $\rightarrow$ (64, 32, 32)
 - First convolution: stride=2 halves spatial dimensions
 - 3 $\rightarrow$ 64 channels (begin feature extraction)
 - **No BatchNorm** on first layer (DCGAN guideline)
 - LeakyReLU activation ($\alpha=0.2$)
3. **D_block 1:** (64, 32, 32) $\rightarrow$ (128, 16, 16) [2x DOWNSAMPLING]
 - Halves spatial size: 32 $\rightarrow$ 16
 - Doubles channels: 64 $\rightarrow$ 128
 - Extracts mid-level features (colors, edges)
4. **D_block 2:** (128, 16, 16) $\rightarrow$ (256, 8, 8) [2x DOWNSAMPLING]
 - Continues downsampling: 16 $\rightarrow$ 8

- Doubles channels: 128→256
 - Captures high-level features (shapes, patterns)
5. **D_block 3:** (256, 8, 8) → (512, 4, 4) [2x DOWNSAMPLING]
 - Final downsampling: 8→4
 - Doubles channels: 256→512
 - Very abstract features (Pokemon-ness, realism)
 6. **Final Classification:** (512, 4, 4) → (1, 1, 1)
 - Convolution with kernel_size=4 eliminates spatial dimensions
 - 512 feature maps → single logit
 - Output: real/fake score (no sigmoid, use BCEWithLogitsLoss)

Key DCGAN Design Principles:

1. **Progressive Downsampling:** 64x64 → 32x32 → 16x16 → 8x8 → 4x4 → 1x1
 - Mirror of generator's upsampling
 - Hierarchical feature extraction
 - Each step halves spatial resolution
2. **Channel Expansion:** 3 → 64 → 128 → 256 → 512 → 1
 - Start with RGB pixels (low-level)
 - Expand to many feature channels (high-level concepts)
 - Opposite of generator (which reduces channels)
3. **Strided Convolutions** (No Pooling):
 - Stride=2 for downsampling instead of max pooling
 - Lets network learn optimal downsampling
 - More parameters = better feature learning
4. **LeakyReLU Throughout:**
 - $\alpha=0.2$ (allows 20% of negative gradients)
 - Prevents dying neurons
 - Better for discriminators than regular ReLU
5. **No BatchNorm on Input Layer:**
 - DCGAN guideline for stability
 - First layer processes raw pixels directly
 - BatchNorm in all subsequent layers
6. **No Sigmoid Output:**
 - Returns raw logits (unnormalized scores)
 - BCEWithLogitsLoss applies sigmoid internally
 - More numerically stable

Parameter Count: approx 2.7 million (similar to Generator) - Slightly more parameters than Generator - Symmetric architecture (mirror image) - Both networks need comparable capacity

Training Role:

1. **Early Training:**
 - Easily spots fake Pokemon (generator is weak)
 - Provides strong gradients for generator improvement
 - Loss decreases quickly
2. **Mid Training:**
 - Generator improves, discriminator must work harder
 - Learns subtle differences (texture quality, color consistency)

- Loss stabilizes around 0.5-1.0

3. Late Training:

- Both networks are equally matched
- Discriminator accuracy → 50% (can barely tell real from fake)
- This equilibrium indicates successful training

Why This Architecture Works: - **Hierarchical Learning:** Low-level (colors) → Mid-level (shapes) → High-level (Pokemon features) - **Sufficient Capacity:** 2.7M parameters can learn complex patterns - **Stable Training:** BatchNorm + LeakyReLU prevent training issues - **Matched Power:** Similar size to Generator (balanced adversarial game)

Comparison to MNIST Discriminator: - **Much Deeper:** 5 layers vs 3 layers (more complex task) - **More Channels:** Up to 512 vs 128 (richer features) - **More Parameters:** 2.7M vs 400K (8x larger) - **Color Input:** RGB vs grayscale (3x more input dimensions)

```
[ ]: # d) Complete Pokemon Discriminator Network

class PokemonDiscriminator(nn.Module):
    """
    DCGAN Discriminator for 64x64 RGB Pokemon images.

    Architecture (mirror of generator):
    Input: (batch, 3, 64, 64) RGB image
    + D_block: (64,64,3) -> (32,32,64)
    + D_block: (32,32,64) -> (16,16,128)
    + D_block: (16,16,128) -> (8,8,256)
    + D_block: (8,8,256) -> (4,4,512)
    + Conv: (4,4,512) -> (1,1,1)          [single output logit]
    """
    def __init__(self, ndf=64):
        super(PokemonDiscriminator, self).__init__()
        # ndf = number of discriminator filters in first conv layer

        # Initial layer (no batch norm on first layer as per DCGAN guidelines)
        self.initial = nn.Sequential(
            nn.Conv2d(3, ndf, kernel_size=4, stride=2, padding=1, bias=False),
            nn.LeakyReLU(0.2, inplace=True)
        )

        # Downsampling blocks
        self.block1 = D_block(ndf, ndf * 2)      # (32,32,64) -> (16,16,128)
        self.block2 = D_block(ndf * 2, ndf * 4)  # (16,16,128) -> (8,8,256)
        self.block3 = D_block(ndf * 4, ndf * 8)  # (8,8,256) -> (4,4,512)

        # Final classification layer
        self.final = nn.Conv2d(ndf * 8, 1, kernel_size=4, stride=1, padding=0,
        ↪ bias=False)
```

```

def forward(self, x):
    """
    Args:
        x: Image tensor of shape (batch, 3, 64, 64)
    Returns:
        Logits of shape (batch, 1, 1, 1)
    """
    x = self.initial(x)    # (batch, 64, 32, 32)
    x = self.block1(x)     # (batch, 128, 16, 16)
    x = self.block2(x)     # (batch, 256, 8, 8)
    x = self.block3(x)     # (batch, 512, 4, 4)
    x = self.final(x)      # (batch, 1, 1, 1)
    return x.view(-1, 1)   # Flatten to (batch, 1)

# Test Pokemon Discriminator
print("Pokemon Discriminator Architecture:")
print("=" * 50)
poke_disc = PokemonDiscriminator(ndf=64).to(device)
test_image = torch.randn(1, 3, 64, 64).to(device)
test_output = poke_disc(test_image)
print(f"Input shape: {test_image.shape}")
print(f"Output shape: {test_output.shape}")
print(f"Total parameters: {sum(p.numel() for p in poke_disc.parameters()),}")
del poke_disc, test_image, test_output

```

[]: # e) Weight Initialization for DCGAN

```

def weights_init_dcgan(m):
    """
    Initialize network weights according to DCGAN paper recommendations.

    - Conv layers: Normal distribution (mean=0, std=0.02)
    - BatchNorm layers: Normal distribution for weights (mean=1, std=0.02), ↵
    ↪ zeros for bias

    This initialization is crucial for stable DCGAN training.
    """
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)

print("Weight Initialization Function Defined:")
print("=" * 50)
print("[OK] weights_init_dcgan: Initializes Conv and BatchNorm layers")

```

```

print(" - Conv weights: Normal(0, 0.02)")
print(" - BatchNorm weights: Normal(1, 0.02)")
print(" - BatchNorm bias: 0")
print("\n[NOTE] Why this matters:")
print(" - Proper initialization prevents vanishing/exploding gradients")
print(" - Standard deviations from DCGAN paper (empirically proven)")
print(" - Critical for GAN convergence")

```

```

[ ]: # Part 2f) Training Function for Pokemon DCGAN
# Task: The training function is provided - explain your understanding of the
      ↪ training block

def train_pokemon_dcgan(netG, netD, dataloader, num_epochs, latent_dim, device,
      ↪ lr=0.0005):
    """
    Training loop for Pokemon DCGAN with real-time visualization.

    Args:
        netG: Generator network
        netD: Discriminator network
        dataloader: Pokemon image dataloader
        num_epochs: Number of training epochs
        latent_dim: Dimension of noise vector
        device: Training device
        lr: Learning rate

    Returns:
        Lists of generator and discriminator losses
    """
    # Loss function
    criterion = nn.BCEWithLogitsLoss()

    # Optimizers (beta1=0.5 is standard for DCGANs)
    optimizerD = optim.Adam(netD.parameters(), lr=lr, betas=(0.5, 0.999))
    optimizerG = optim.Adam(netG.parameters(), lr=lr, betas=(0.5, 0.999))

    # Fixed noise for visualization
    fixed_noise = torch.randn(21, latent_dim, 1, 1).to(device)

    # Tracking
    G_losses = []
    D_losses = []

    print("Starting Pokemon DCGAN Training...")
    print("=" * 70)

    for epoch in range(1, num_epochs + 1):

```

```

epoch_start = time.time()
epoch_G_loss = 0
epoch_D_loss = 0
num_batches = 0

for i, (real_images, _) in enumerate(dataloader):
    batch_size = real_images.size(0)
    real_images = real_images.to(device)

    # =====
    # Train Discriminator
    # =====
    netD.zero_grad()

    # Real images
    real_labels = torch.ones(batch_size, 1).to(device)
    real_output = netD(real_images)
    d_loss_real = criterion(real_output, real_labels)

    # Fake images
    noise = torch.randn(batch_size, latent_dim, 1, 1).to(device)
    fake_images = netG(noise)
    fake_labels = torch.zeros(batch_size, 1).to(device)
    fake_output = netD(fake_images.detach())
    d_loss_fake = criterion(fake_output, fake_labels)

    # Total discriminator loss
    d_loss = d_loss_real + d_loss_fake
    d_loss.backward()
    optimizerD.step()

    # =====
    # Train Generator
    # =====
    netG.zero_grad()

    # Generator wants discriminator to think fakes are real
    fake_output = netD(fake_images)
    g_loss = criterion(fake_output, real_labels)
    g_loss.backward()
    optimizerG.step()

    # Track losses
    epoch_G_loss += g_loss.item()
    epoch_D_loss += d_loss.item()
    num_batches += 1

```

```

    # Average losses
    avg_G_loss = epoch_G_loss / num_batches
    avg_D_loss = epoch_D_loss / num_batches
    G_losses.append(avg_G_loss)
    D_losses.append(avg_D_loss)

    epoch_time = time.time() - epoch_start

    # Print progress
    print(f"Epoch [{epoch}/{num_epochs}] | "
          f"D_loss: {avg_D_loss:.4f} | "
          f"G_loss: {avg_G_loss:.4f} | "
          f"Time: {epoch_time:.2f}s")

    # Generate samples for visualization
    if epoch % 5 == 0 or epoch == 1:
        netG.eval()
        with torch.no_grad():
            fake_samples = netG(fixed_noise)
            # Denormalize for display
            fake_samples = (fake_samples + 1) / 2

            # Create grid display
            grid = make_grid(fake_samples[:16], nrow=4, padding=2,
                               normalize=False)

            plt.figure(figsize=(8, 8))
            plt.imshow(grid.permute(1, 2, 0).cpu().numpy())
            plt.title(f'Generated Pokemon - Epoch {epoch}')
            plt.axis('off')
            plt.tight_layout()
            plt.savefig(f'pokemon_generated/epoch_{epoch:04d}.png')
            plt.show()
        netG.train()

    print("=" * 70)
    print("Pokemon DCGAN Training Complete!")

    return G_losses, D_losses

print("Pokemon DCGAN Training Function Defined:")
print("=" * 50)
print("[OK] train_pokemon_dcgan: Complete training loop with visualization")
print(" - Generates sample images every 5 epochs")
print(" - Tracks and returns loss history")
print(" - Uses DCGAN-specific hyperparameters (beta1=0.5)")

```

1.5.7 [SUMMARY] Summary - Part 2 Classes and Functions Complete

What we accomplished: - [DONE] **G_block**: Modular generator block with TransposedConv + BatchNorm + ReLU - [DONE] **D_block**: Modular discriminator block with Conv + BatchNorm + LeakyReLU - [DONE] **PokemonGenerator**: Complete 64x64 RGB image generator - Progressive upsampling: $1 \times 1 \rightarrow 4 \times 4 \rightarrow 8 \times 8 \rightarrow 16 \times 16 \rightarrow 32 \times 32 \rightarrow 64 \times 64$ - Channel reduction: $512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 3$ (RGB)

- [DONE] **PokemonDiscriminator**: Complete image classifier
 - Progressive downsampling: $64 \times 64 \rightarrow 32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8 \rightarrow 4 \times 4 \rightarrow 1 \times 1$
 - Channel expansion: $3 \text{ (RGB)} \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1 \text{ (logit)}$
- [DONE] **DCGAN Weight Initialization**: Proper initialization for stable training
- [DONE] **Training Loop**: Complete with real-time visualization

Key Learning Points:

1. Modular Architecture:

- Building blocks make architecture flexible and readable
- Easy to adjust depth by adding/removing blocks
- Generator and discriminator are symmetric (mirror images)

2. DCGAN Innovations:

- All-convolutional (no pooling/unpooling layers)
- Batch normalization in all layers (except input/output)
- ReLU in generator, LeakyReLU in discriminator
- Tanh output for generator (matches $[-1, 1]$ normalized data)

3. Architectural Relationships:

Generator: Noise $\rightarrow$ Expand channels $\rightarrow$ Upsample $\rightarrow$ Reduce channels $\rightarrow$ RGB

Discriminator: RGB $\rightarrow$ Expand channels $\rightarrow$ Downsample $\rightarrow$ Compress $\rightarrow$ Single value

4. Why LeakyReLU in Discriminator?:

- Regular ReLU can “die” (always output 0)
- LeakyReLU allows small negative gradients ($\alpha=0.2$)
- Helps discriminator learn from negative features
- More stable training

Architecture Comparison:

MNIST GAN: 28x28x1 grayscale, simpler architecture

Pokemon DCGAN: 64x64x3 RGB, deeper network, modular blocks

Next Step: Load Pokemon dataset and execute training

1.6 Section 6: Part 2 - Pokemon Data Loading and Training Execution

1.6.1 Purpose:

Load the Pokemon dataset, create normalization function, initialize DCGAN models, and execute training.

1.6.2 Dataset Information:

- **Pokemon Dataset:** Collected Pokemon sprites from multiple generations
- **Image Size:** 64x64 RGB
- **Normalization:** Scale to $[-1, 1]$ to match generator's tanh output

```
[ ]: # Part 2a) Download and prepare Pokemon dataset
# Task: Read Pokemon dataset (40,597 images from 71 classes)
# Create normalization function: divide by 255, apply Gaussian normalization
↳ (mean=0.5, std=0.5)

def download_pokemon_dataset(data_dir='./pokemon_data'):
    """
    Download Pokemon dataset directly without d2l dependency. D2L uses python 3.
    ↳ 9 which is not supported

    Dataset contains Pokemon sprites organized by Pokemon ID.
    """
    url = 'https://d2l-data.s3-accelerate.amazonaws.com/pokemon.zip'
    zip_path = os.path.join(data_dir, 'pokemon.zip')
    extract_path = data_dir

    # Create directory if it doesn't exist
    os.makedirs(data_dir, exist_ok=True)

    # Check if already downloaded
    if os.path.exists(os.path.join(extract_path, 'pokemon')):
        print(f"Pokemon dataset already exists at: {extract_path}")
        return extract_path

    # Download the dataset
    print(f"Downloading Pokemon dataset from {url}...")
    response = requests.get(url, stream=True)
    total_size = int(response.headers.get('content-length', 0))

    with open(zip_path, 'wb') as f:
        if total_size == 0:
            f.write(response.content)
        else:
            downloaded = 0
            for chunk in response.iter_content(chunk_size=8192):
                downloaded += len(chunk)
                f.write(chunk)
                done = int(50 * downloaded / total_size)
                print(f"\r[ '=' * done]{' ' * (50-done)} {downloaded}/
↳ {total_size} bytes", end='')
            print("\n[OK] Download complete!")
```

```

# Extract the zip file
print("Extracting dataset...")
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)
print(f"[OK] Dataset extracted to: {extract_path}")

# Clean up zip file
os.remove(zip_path)
print("[OK] Cleaned up zip file")

return extract_path

# Download dataset
data_dir = download_pokemon_dataset('./pokemon_data')

print("\n[NOTE] Dataset Information:")
print(" - Images normalized to [-1, 1] range")

print(" - Normalization: (pixel/255 - 0.5) / 0.5")
print(" - Matches the tanh output range of the generator!")

```

[]: # b) Create PyTorch Dataset and DataLoader for Pokemon

```

from PIL import Image
import os

class PokemonDataset(Dataset):
    """Custom Dataset for Pokemon images."""
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.image_paths = []

        # Collect all image paths
        pokemon_dir = os.path.join(root_dir, 'pokemon')
        for pokemon_id in os.listdir(pokemon_dir):
            pokemon_folder = os.path.join(pokemon_dir, pokemon_id)
            if os.path.isdir(pokemon_folder):
                for img_name in os.listdir(pokemon_folder):
                    if img_name.endswith(('.png', '.jpg', '.jpeg')):
                        self.image_paths.append(os.path.join(pokemon_folder,
↪img_name))

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):

```

```

        img_path = self.image_paths[idx]
        image = Image.open(img_path).convert('RGB')

        if self.transform:
            image = self.transform(image)

        return image, 0 # Return dummy label for compatibility

# Define transformations
pokemon_transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(), # Converts to [0, 1] and (C, H, W)
    transforms.Lambda(lambda x: (x - 0.5) / 0.5) # Normalize to [-1, 1]
])

# Create dataset and dataloader
pokemon_dataset = PokemonDataset(data_dir, transform=pokemon_transform)
POKEMON_BATCH_SIZE = 256

pokemon_loader = DataLoader(
    pokemon_dataset,
    batch_size=POKEMON_BATCH_SIZE,
    shuffle=True,
    num_workers=2,
    pin_memory=True if torch.cuda.is_available() else False,
    drop_last=True # Drop last incomplete batch for stable training
)

print("Pokemon Dataset Loaded:")
print("=" * 50)
print(f"Total images: {len(pokemon_dataset):,}")
print(f"Batch size: {POKEMON_BATCH_SIZE}")
print(f"Number of batches: {len(pokemon_loader)}")
print(f"Image shape: (3, 64, 64) RGB")
print(f"Value range: [-1, 1]")

```

```

[ ]: # Part 2b) Visualize Pokemon images
# Task: Run provided code to visualize data and get better insights

print("Visualizing Sample Pokemon Images:")
print("=" * 50)

# Get a batch of images
sample_batch, _ = next(iter(pokemon_loader))

# Denormalize for visualization
sample_images = (sample_batch[:20] + 1) / 2 # [-1, 1] -> [0, 1]

```

```

# Create grid
fig, axes = plt.subplots(4, 5, figsize=(12, 10))
for i, ax in enumerate(axes.flat):
    # Convert from (C, H, W) to (H, W, C) for display
    img = sample_images[i].permute(1, 2, 0).cpu().numpy()
    ax.imshow(img)
    ax.axis('off')

plt.suptitle('Sample Pokemon Images (Real Data)', fontsize=14,
             fontweight='bold')
plt.tight_layout()
plt.show()

print("[OK] Dataset contains diverse Pokemon sprites")
print("[OK] Images are 64x64 RGB")
print("[OK] Ready for DCGAN training")

```

```

[ ]: # d) Initialize Pokemon DCGAN models

LATENT_DIM = 100
NGF = 64 # Number of generator filters
NDF = 64 # Number of discriminator filters
LEARNING_RATE_POKEMON = 0.0005

# Create models
netG = PokemonGenerator(noise_dim=LATENT_DIM, ngf=NGF).to(device)
netD = PokemonDiscriminator(ndf=NDF).to(device)

# Apply DCGAN weight initialization
netG.apply(weights_init_dcgan)
netD.apply(weights_init_dcgan)

print("Pokemon DCGAN Models Initialized:")
print("=" * 50)
print(f"Generator parameters: {sum(p.numel() for p in netG.parameters()):,}")
print(f"Discriminator parameters: {sum(p.numel() for p in netD.parameters()):,}")
print(f"Weights initialized with DCGAN scheme")
print(f"Device: {device}")

# Test with sample input
print("\nTesting Generator:")
test_z = torch.randn(1, LATENT_DIM, 1, 1).to(device)
with torch.no_grad():
    test_gen_output = netG(test_z)
print(f"Input: {test_z.shape} → Output: {test_gen_output.shape}")

```

```

print("\nTesting Discriminator:")
with torch.no_grad():
    test_disc_output = netD(test_gen_output)
print(f"  Input: {test_gen_output.shape} → Output: {test_disc_output.shape}")
print("\n[OK] Models ready for training!")

```

```

[ ]: # Part 2f) Train Pokemon DCGAN
# Task: Call the train function with appropriate values

NUM_EPOCHS_POKEMON = 40 # Can increase to 100+ for better results

print("Starting Pokemon DCGAN Training")
print("=" * 70)
print(f"Configuration:")
print(f"  - Epochs: {NUM_EPOCHS_POKEMON}")
print(f"  - Batch size: {POKEMON_BATCH_SIZE}")
print(f"  - Learning rate: {LEARNING_RATE_POKEMON}")
print(f"  - Latent dimension: {LATENT_DIM}")
print(f"  - Device: {device}")
print("=" * 70)
print("\n[hourglass] Training in progress... This may take 20-60 minutes,
↳ depending on hardware.\n")

# Train the DCGAN
G_losses_pokemon, D_losses_pokemon = train_pokemon_dcgan(
    netG=netG,
    netD=netD,
    dataloader=pokemon_loader,
    num_epochs=NUM_EPOCHS_POKEMON,
    latent_dim=LATENT_DIM,
    device=device,
    lr=LEARNING_RATE_POKEMON
)

```

1.6.3 [THINKING] Part 2f) Understanding the Training Block

Assignment Task: Explain your understanding about the training block. What is your understanding from the graph?

Explanation of the Training Process:

1. Weight Initialization:

- All convolutional layers initialized with Normal(0, 0.02)
- Batch normalization weights: Normal(1, 0.02)
- Critical for preventing gradient issues in deep networks

2. Training Loop:

- **Discriminator Update:**
 - Processes real Pokemon images → should output approx1

- Processes generated (fake) images → should output approx 0
- Backpropagates total loss to improve classification
- **Generator Update:**
 - Generates fake Pokemon from random noise
 - Discriminator evaluates them
 - Generator learns to fool discriminator (wants output approx 1)
 - Only generator weights updated (discriminator frozen)
- 3. **Optimization:**
 - Adam optimizer with $\beta_1=0.5$ (lower than typical 0.9)
 - Helps with GAN training stability
 - Learning rate: 0.0005
- 4. **Expected Behavior:**
 - Early epochs: Blurry, color-blob images
 - Middle epochs: Pokemon-like shapes emerge - The goal is stability, not necessarily low loss values

Understanding from the Loss Graph: - Late epochs: Recognizable Pokemon features (eyes, limbs, patterns) - Oscillations are normal and expected in GAN training

- Loss values alone don't indicate quality - must inspect generated images

The loss curves show the adversarial competition between generator and discriminator: - The adversarial game reaches equilibrium when both networks are equally matched

4. What the Graph Tells Us:

5. Discriminator Loss (Red):

- Represents how well D distinguishes real from fake - Visual quality improves even if losses fluctuate
- When D loss is low (approx 0.5-1.0): D is correctly classifying most images - Losses stabilize over time (convergence)
- When D loss spikes: G has temporarily fooled D with better fakes - Neither network dominates (no collapse)
- Both losses oscillate but remain relatively balanced

6. Generator Loss (Blue):

- Represents how well G fools D
- When G loss is low: G successfully tricks D (fake images classified as real) - When G loss is high: D detects fakes easily, G needs improvement

1.7 Section 7: Part 2 - Results Analysis and Final Summary

1.7.1 Purpose:

Analyze the trained DCGAN performance, visualize loss curves, and summarize key learnings.

```
[ ]: # Part 2f continued) Plot Pokemon DCGAN training loss curves
# Task: Analyze the loss graph and explain understanding

plt.figure(figsize=(10, 5))
plt.plot(G_losses_pokemon, label='Generator Loss', linewidth=2, alpha=0.8,
         color='blue')
plt.plot(D_losses_pokemon, label='Discriminator Loss', linewidth=2, alpha=0.8,
         color='red')
plt.xlabel('Epoch', fontsize=12)
plt.ylabel('Loss', fontsize=12)
plt.title('Pokemon DCGAN Training Loss Curves', fontsize=14, fontweight='bold')
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("Loss Curve Analysis:")
print("=" * 50)
print(f"Final Generator Loss: {G_losses_pokemon[-1]:.4f}")
print(f"Final Discriminator Loss: {D_losses_pokemon[-1]:.4f}")
print("\n[NOTE] What to look for in Pokemon DCGAN:")
print(" - Both losses should show general stability")
print(" - More oscillation than MNIST is normal (more complex images)")
print(" - D loss should stay roughly between 0.5-2.0")
print(" - G loss typically higher than MNIST GAN")
print(" - Visual quality matters more than loss values!")
```

```
[ ]: # b) Generate final Pokemon samples

print("Generating Final Pokemon Samples:")
print("=" * 50)

# Generate a batch of Pokemon
netG.eval()
num_samples = 25
final_noise = torch.randn(num_samples, LATENT_DIM, 1, 1).to(device)

with torch.no_grad():
    final_pokemon = netG(final_noise)
    # Denormalize from [-1, 1] to [0, 1]
    final_pokemon = (final_pokemon + 1) / 2

# Display the generated Pokemon
fig, axes = plt.subplots(5, 5, figsize=(12, 12))
for i, ax in enumerate(axes.flat):
    img = final_pokemon[i].permute(1, 2, 0).cpu().numpy()
    ax.imshow(img)
```

```

        ax.axis('off')

plt.suptitle('Generated Pokemon - Final Results', fontsize=14,
            ↪fontweight='bold')
plt.tight_layout()
plt.show()

print("\n[OK] Generated 25 unique Pokemon-like creatures!")
print("[OK] Notice color variations, shapes, and features")
print("[OK] Some may look more realistic than others (normal for GANs)")

```

```

[ ]: # c) Compare real vs generated Pokemon side-by-side

print("Real vs Generated Pokemon Comparison:")
print("=" * 50)

# Get real Pokemon samples
real_samples, _ = next(iter(pokemon_loader))
real_samples = (real_samples[:8] + 1) / 2 # Denormalize

# Generate fake Pokemon
fake_noise = torch.randn(8, LATENT_DIM, 1, 1).to(device)
with torch.no_grad():
    fake_samples = netG(fake_noise)
    fake_samples = (fake_samples + 1) / 2

# Display side by side
fig, axes = plt.subplots(2, 8, figsize=(16, 4))

# Real images (top row)
for i in range(8):
    img = real_samples[i].permute(1, 2, 0).cpu().numpy()
    axes[0, i].imshow(img)
    axes[0, i].axis('off')
    if i == 0:
        axes[0, i].set_ylabel('Real', fontsize=12, rotation=0, labelpad=30)

# Generated images (bottom row)
for i in range(8):
    img = fake_samples[i].permute(1, 2, 0).cpu().numpy()
    axes[1, i].imshow(img)
    axes[1, i].axis('off')
    if i == 0:
        axes[1, i].set_ylabel('Generated', fontsize=12, rotation=0, labelpad=30)

plt.suptitle('Real vs Generated Pokemon Comparison', fontsize=14,
            ↪fontweight='bold')

```

```
plt.tight_layout()
plt.show()

print("\n[NOTE] Observations:")
print(" - Generator learned color schemes typical of Pokemon")
print(" - Spatial structure (centered creatures) is preserved")
print(" - Some generated Pokemon have coherent features")
print(" - Artifacts and blur are common (GAN limitation)")
```

1.7.2 [SUMMARY] Summary - Part 2 Complete

What we accomplished in Part 2: - [DONE] Implemented DCGAN architecture with modular building blocks - [DONE] Downloaded and prepared Pokemon dataset (64x64 RGB images) - [DONE] Applied proper DCGAN weight initialization - [DONE] Trained generator to create Pokemon-like creatures from noise - [DONE] Analyzed training dynamics and loss curves - [DONE] Generated diverse Pokemon samples

Key Learning Achievements:

1. **DCGAN Architecture:**
 - **Modular design:** G_block and D_block for flexible architecture
 - **All-convolutional:** No pooling or fully-connected hidden layers
 - **Batch normalization:** Stabilizes training in deep networks
 - **Proper activations:** ReLU in generator, LeakyReLU in discriminator
2. **Differences from Basic GAN (Part 1):**
 - **Scale:** 64x64 RGB vs 28x28 grayscale
 - **Complexity:** Deeper network (4 blocks vs 2-3 layers)
 - **Data:** More complex (Pokemon vs digits)
 - **Training:** Requires more careful initialization and tuning
3. **Training Insights:**
 - **Weight initialization critical:** Normal(0, 0.02) for stability
 - **Beta1=0.5:** Lower momentum helps GAN convergence
 - **Visualization essential:** Losses don't tell full story
 - **Mode diversity:** Some generated samples better than others
4. **Challenges Observed:**
 - **Mode collapse risk:** Generator might favor certain Pokemon styles
 - **Artifacts:** Some blur and distortion in generated images
 - **Training time:** Significantly longer than MNIST
 - **Hyperparameter sensitivity:** Learning rate, architecture choices matter

Aspect	MNIST GAN (Part 1)	Pokemon DCGAN (Part 2)
Image Size	28x28x1	64x64x3
Architecture	Basic CNN	DCGAN with blocks
Dataset Size	60,000 images	approx 10,000 images
Training Time	10-30 min	20-60 min
Complexity	Simple patterns	Complex structures
Stability	More stable	More challenging

1.8 [graduation] Final Summary: Assignment 6 Complete

1.8.1 Overall Accomplishments:

Technical Skills Developed: 1. [DONE] Built GANs from scratch in PyTorch 2. [DONE] Implemented both basic and DCGAN architectures 3. [DONE] Trained on grayscale (MNIST) and RGB (Pokemon) datasets 4. [DONE] Applied proper weight initialization techniques 5. [DONE] Monitored and debugged adversarial training 6. [DONE] Visualized training progress and results

Key Theoretical Concepts Mastered:

1. Adversarial Training:

- Two networks competing in a minimax game
- Generator learns data distribution implicitly
- Discriminator provides learning signal through gradients

2. Architecture Design:

- Transposed convolutions for upsampling (generator)
- Standard convolutions for downsampling (discriminator)
- Batch normalization for training stability
- Activation function choices (ReLU vs LeakyReLU)

3. Training Dynamics:

- Balance between generator and discriminator power
- Loss values vs visual quality assessment
- Common failure modes (mode collapse, instability)
- Hyperparameter importance

4. PyTorch Implementation:

- Custom nn.Module classes
- Efficient data loading with DataLoader
- GPU acceleration
- Checkpoint management

1.8.2 Real-World Applications:

GANs are used for: - **Image Generation:** Art, design, synthetic data - **Image-to-Image Translation:** Style transfer, colorization, super-resolution - **Data Augmentation:** Creating training data for other models - **Anomaly Detection:** Identifying unusual patterns - **Drug Discovery:** Generating molecular structures - **Fashion/Game Design:** Creating new designs and characters

1.8.3 What is DCGAN?

DCGAN = Deep Convolutional Generative Adversarial Network

DCGAN introduced specific architectural guidelines that make GANs more stable: 1. **Replace pooling with strided convolutions** (discriminator) and transposed convolutions (generator) 2. **Use batch normalization** in both generator and discriminator 3. **Remove fully connected hidden layers** (all-convolutional architecture) 4. **Use ReLU in generator** (except output layer with Tanh) 5. **Use LeakyReLU in discriminator** (prevents dying neurons)

These principles significantly improved GAN training stability compared to earlier GANs!

1.8.4 Ways to Improve Pokemon DCGAN Results:

1. Training Improvements (Easy to Implement): - **More epochs:** Increase from 40 to 100-200 epochs - **Learning rate scheduling:** Reduce learning rate gradually `python scheduler_G = optim.lr_scheduler.StepLR(optimizerG, step_size=50, gamma=0.5)`
`scheduler_D = optim.lr_scheduler.StepLR(optimizerD, step_size=50, gamma=0.5)` - **Label smoothing:** Use 0.9 instead of 1.0 for real labels (reduces discriminator overconfidence) - **Different learning rates:** Try `lr_D=0.0004`, `lr_G=0.0001` (slower discriminator)

2. Architecture Improvements (Moderate): - **Spectral normalization:** Stabilizes discriminator training - **Self-attention layers:** Better captures long-range dependencies - **Deeper networks:** Add more G_block/D_block layers - **Residual connections:** Skip connections for gradient flow

3. Advanced Techniques (More Complex): - **Progressive Growing:** Start with 8x8, gradually increase to 64x64 - **Wasserstein GAN (WGAN-GP):** Better loss function, more stable - **StyleGAN architecture:** State-of-the-art for high-quality images - **Conditional GAN:** Control Pokemon type/color during generation - **Data augmentation:** Flip, rotate, color jitter Pokemon images

4. Dataset Improvements: - **More data:** Collect additional Pokemon images - **Better pre-processing:** Remove backgrounds, center Pokemon - **Consistent sizing:** Ensure all images are properly scaled

Quick Win to Try Now:

```
# In the training cell, change NUM_EPOCHS_POKEMON from 40 to 100
NUM_EPOCHS_POKEMON = 100
```

```
# And modify learning rates
LEARNING_RATE_POKEMON = 0.0002 # Reduce from 0.0005
```

Expected Improvement: With 100 epochs and lower learning rate, you should see: - Sharper edges on generated Pokemon - More consistent color schemes - Better defined features (eyes, limbs) - Less color bleeding and artifacts

Remember: Even with improvements, Pokemon generation is harder than MNIST. The results will never be photo-realistic with basic DCGAN, but they should show clear Pokemon-like characteristics!

1.8.5 Final Thoughts:

This assignment demonstrated that GANs can learn complex data distributions without explicit supervision. The generator never sees real images directly—it only receives gradients from the discriminator. This adversarial process creates a powerful learning paradigm that has revolutionized generative modeling in computer vision.

The journey from random noise to realistic images showcases the power of deep learning and adversarial training!